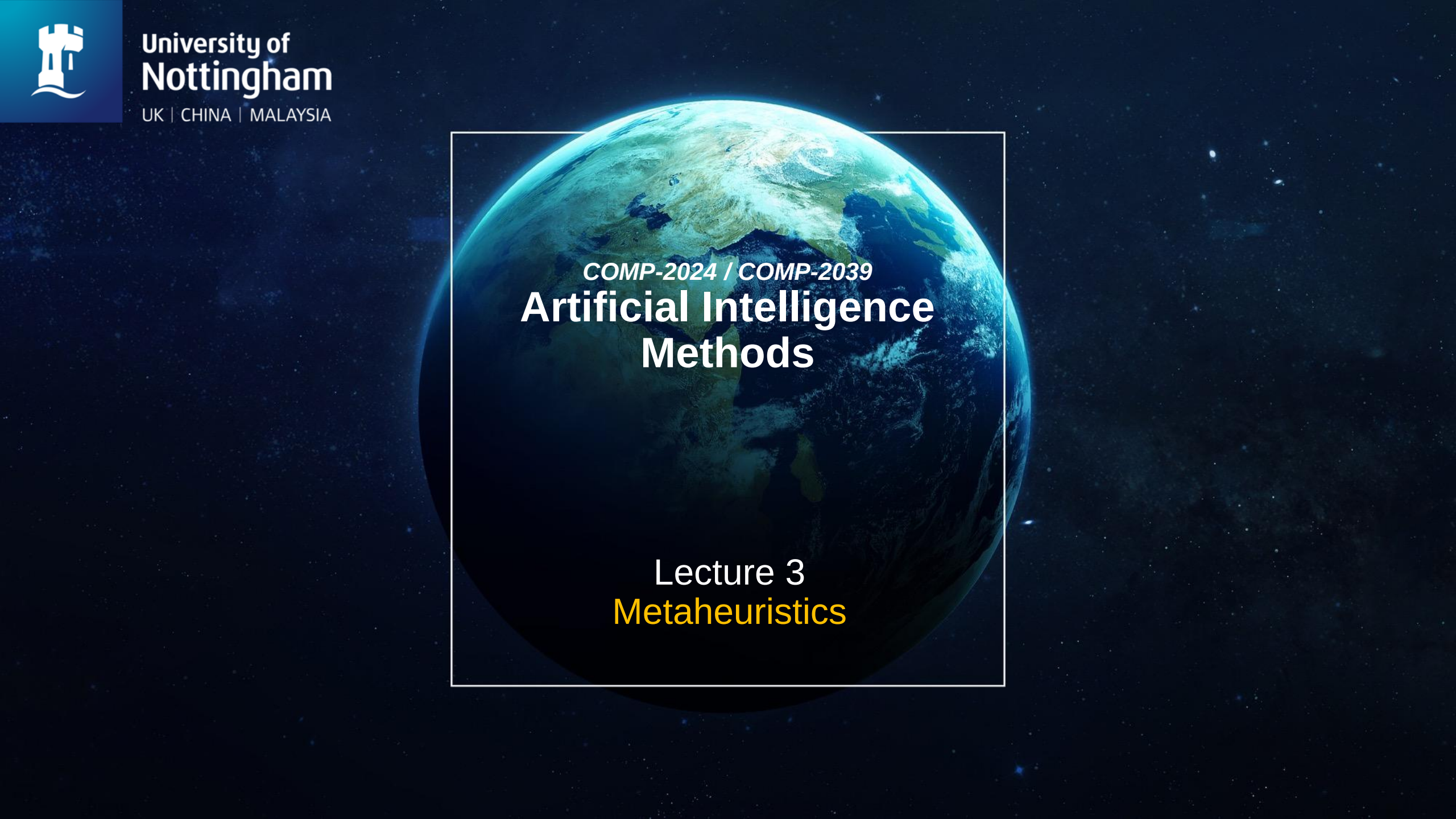

University of
Nottingham
UK | CHINA | MALAYSIA
COMP-2024 / COMP-2039
Artificial Intelligence Methods
Lecture 3
Metaheuristics

Learning Outcomes

IDENTIFY

1. Metaheuristics
2. Local Search Metaheuristics and Iterated Local Search
3. Tabu Search
4. Scheduling

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

University of Nottingham
UK | CHINA | MALAYSIA

Metaheuristics

What is a Metaheuristic?

A **metaheuristic** is a *high-level problem independent* algorithmic framework that provides a set of *guidelines* or *strategies* to develop heuristic *optimisation* algorithms

K. Sörensen and F. Glover. Metaheuristics. In S.I. Gass and M. Fu, editors, Encyclopedia of Operations Research and Management Science, pp 960–970. Springer, New York, 2013.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

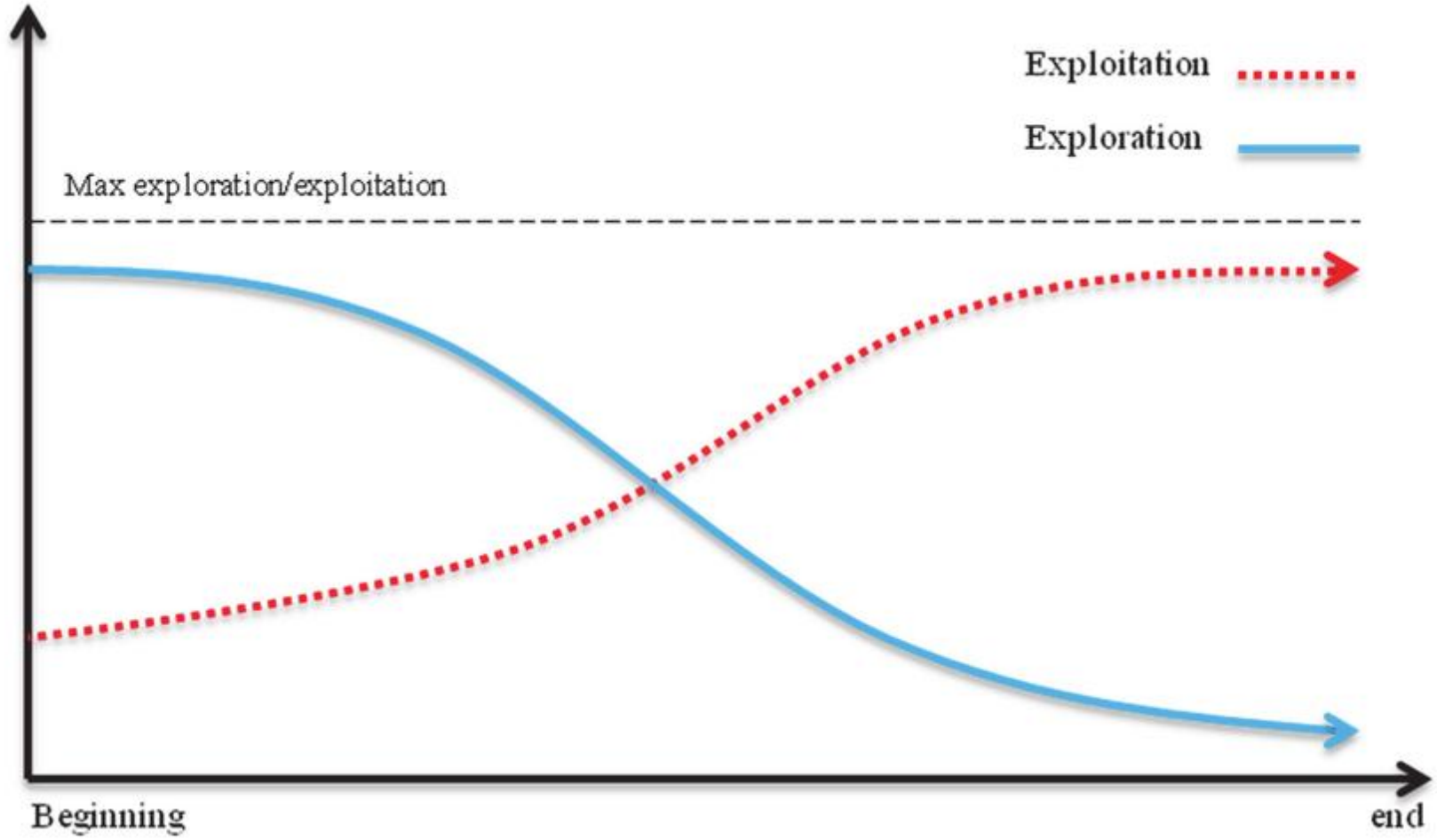
More precisely

- **Metaheuristics** are *high-level problem-solving frameworks* designed to find *approximate solutions* to complex optimization problems within a *reasonable time*.

- A metaheuristic method is a method that *sits on top of a local search algorithm to change its behavior*.

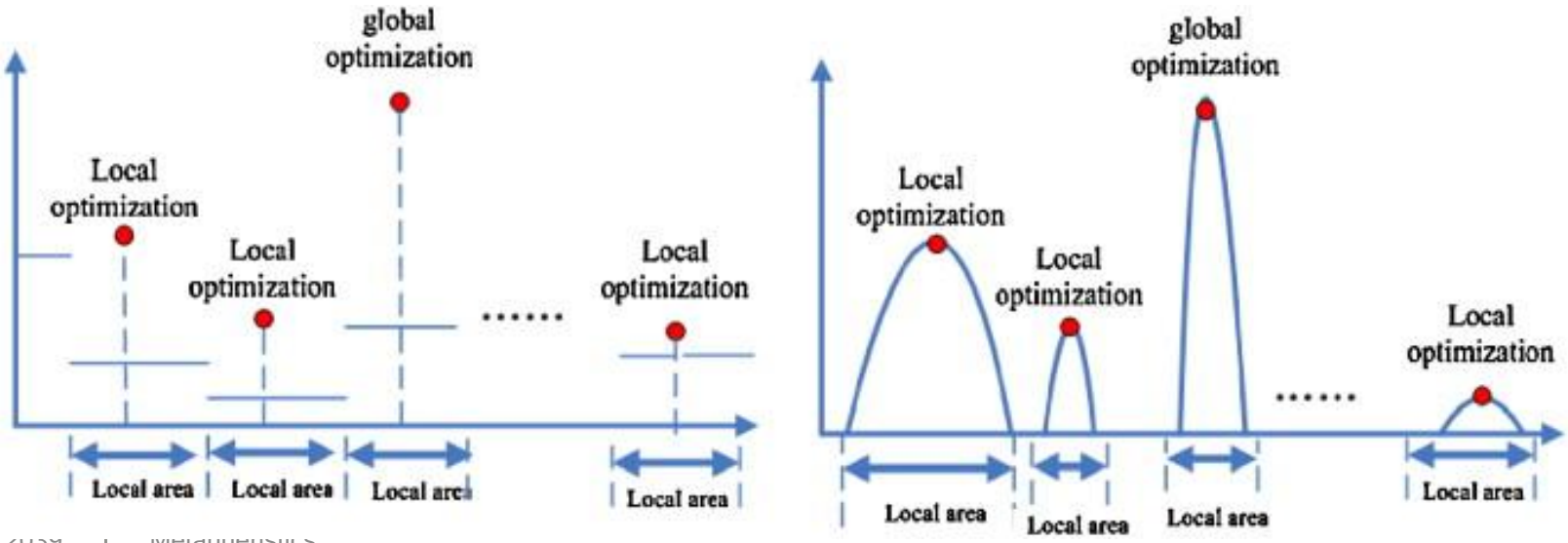
- They guide and refine the search process using strategies that balance **exploration** (searching new areas) and **exploitation** (refining known good areas).

Exploration vs Exploitation



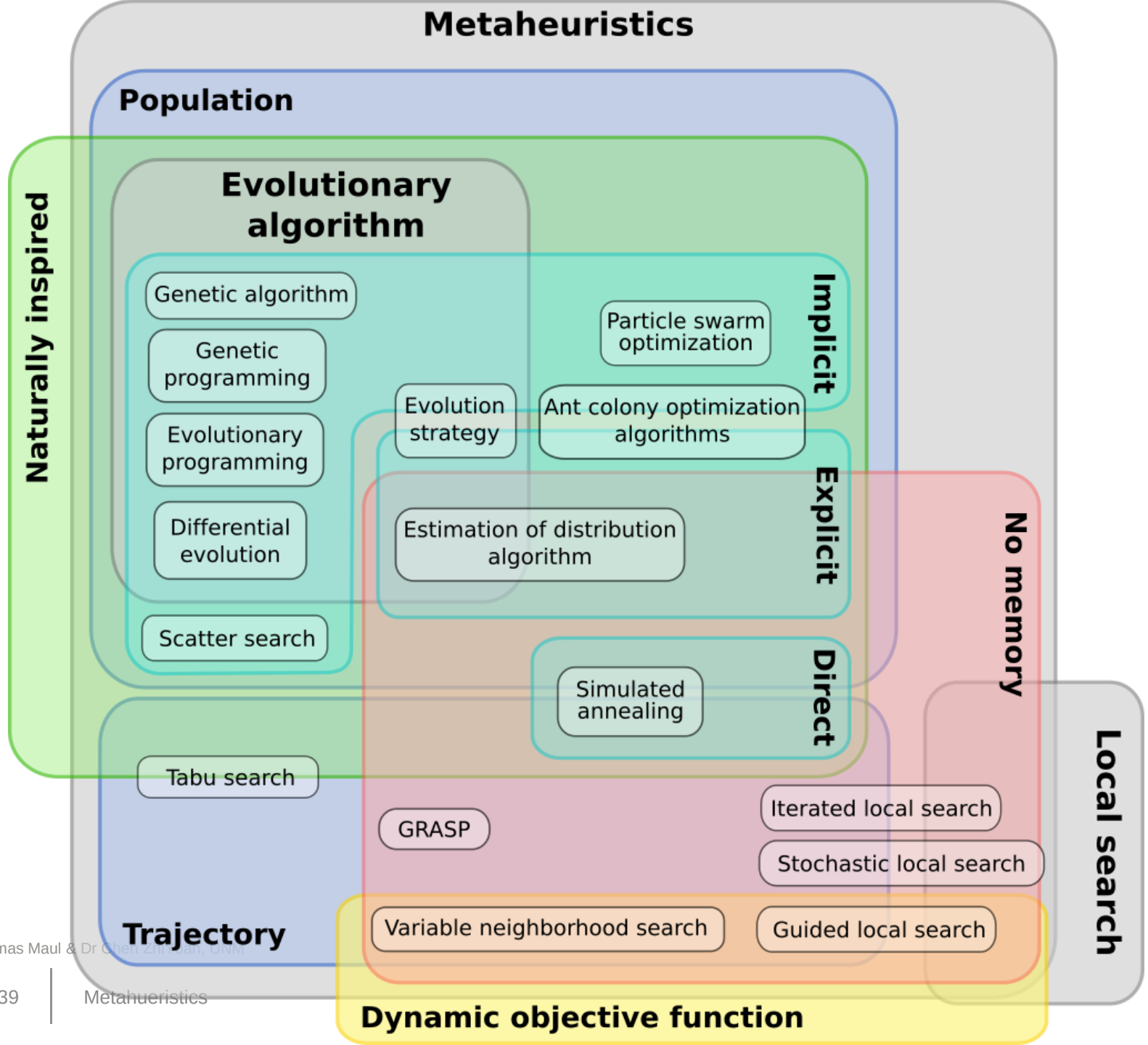
Key Concepts

- **Search Space**: The set of all possible solutions.
- **Exploration**: Searching broadly across the solution space.
- **Exploitation**: Refining the best solutions found so far.
- **Objective Function**: The function being optimized (*minimized* or *maximized*).



Characteristics of Metaheuristics

- **General Applicability**: Can be applied to a wide range of problems without major modifications.
- **Approximate Solutions**: Do not guarantee the optimal solution but aim to find a good solution within practical constraints.
- **Hybridizable**: Can be combined with other optimization methods.
- **Stochastic**: Often involve randomness to explore the search space.



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen Zhenyan, UNM

Main Components of a Metaheuristic Search Method

- **Representation** of candidate solutions
- **Evaluation function**
- **Initialisation**: e.g., initial candidate solution may be chosen
  - randomly
  - use a constructive heuristic
  - according to some regular pattern
  - based on other information (e.g. results of a prior search), and more
- **Neighbourhood relation** (**move operators**)
  - Search process (guideline)
  - Stopping conditions
- **Mechanism for escaping from local optima**

ACK: [illegible]

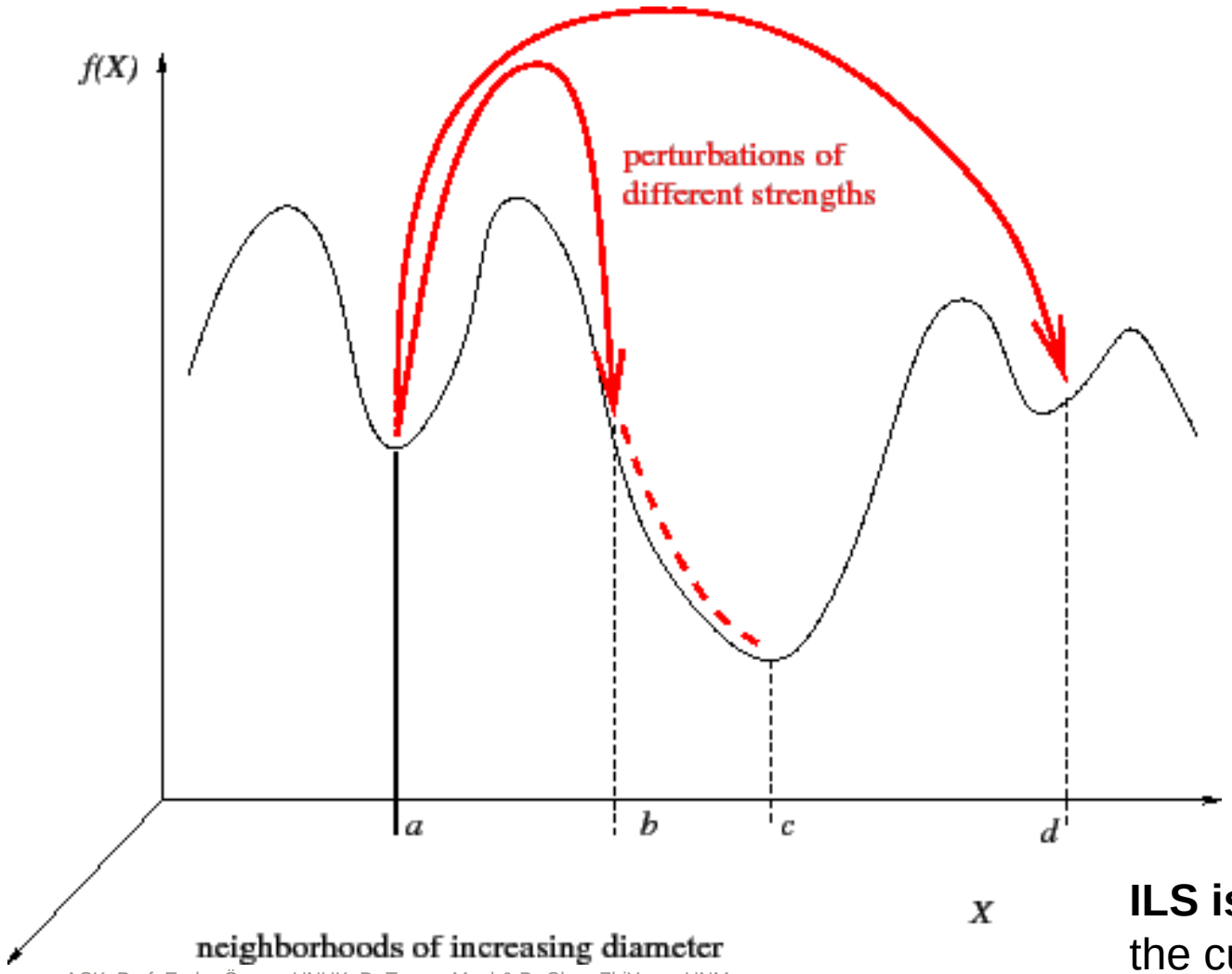
Mechanisms for Escaping from Local Optima I

- Search process (guideline)
- Stopping conditions
- *Mechanism for escaping from local optima*

- Iterate with different solutions, or **restart** (re-initialise search whenever a local optimum is encountered) → mutate
  - Initialisation could be costly
  - E.g., **Iterated Local Search**, *Greedy Randomized Adaptive Search Procedure* (GRASP)
- **Change the search landscape**
  - Change the objective function (E.g., *Guided Local Search*)
  - Use (mix) different neighbourhoods (E.g., *Variable Neighbourhood Search*, Hyper-heuristics)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Random Restart: Iterated Local Search (more later…)

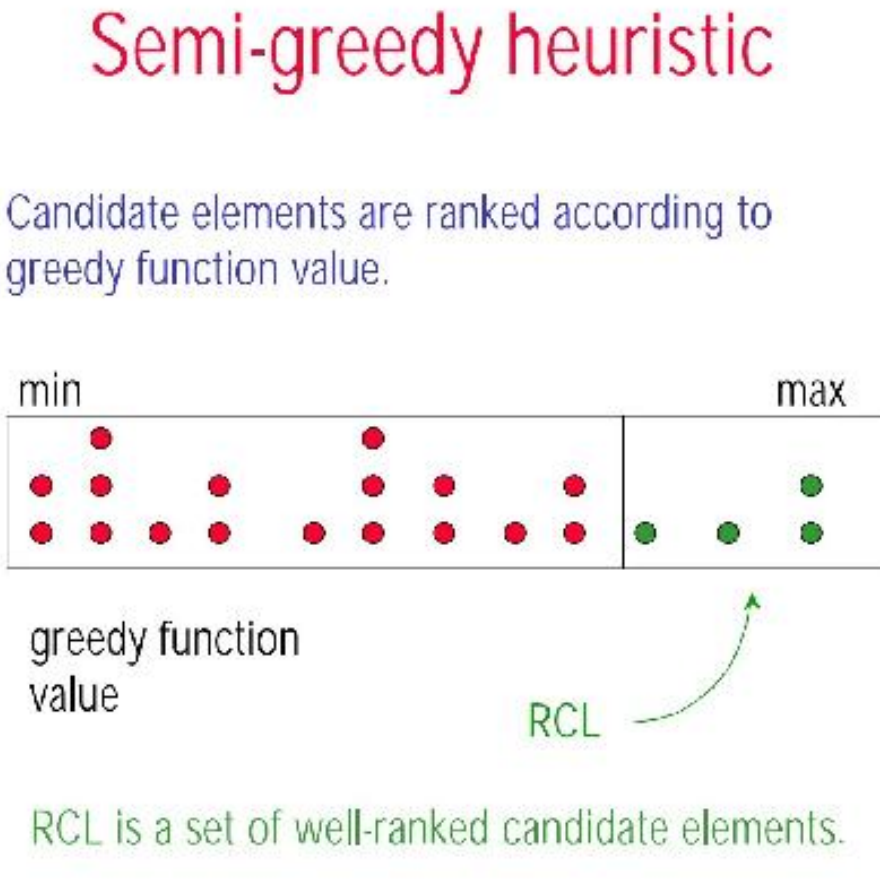


1. **Local Search**: This involves exploring the solution space by starting from an initial solution and iteratively moving to neighboring solutions. Local search can get stuck in local optima.
2. **Perturbation**: To escape local optima, ILS introduces a perturbation mechanism that modifies the current solution significantly, allowing the search to continue in a different direction.
3. **Iterative Process**: The algorithm repeatedly applies local search and perturbation to refine solutions and explore new areas of the solution space.

ILS is memoryless in the sense that it only keeps track of the current and best-found solution but does not maintain a memory of past searches (like Tabu Search does).

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Random Restart: Greedy randomized adaptive search procedure



GRASP: Tries to Combine the Advantages of Random and Greedy Solution Construction Together.

1. **Greedy Construction:** The algorithm builds a solution incrementally, selecting components based on a greedy criterion, but with a random element that introduces diversity.
2. **Restricted Candidate List (RCL)**: Instead of choosing the best candidate at each step, GRASP maintains a list of the *best candidates* (RCL) (top-k). A random choice is made from this list, allowing for exploration of different paths.
3. **Local Search**: After constructing a solution, GRASP applies a local search to refine it. This step aims to improve the solution by exploring its neighborhood.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Guided Local Search

- **Guided Local Search (GLS)** is a metaheuristic optimization algorithm that enhances local search methods by guiding them out of local optima using a penalty-based mechanism. It is particularly useful for combinatorial optimization problems, such as MAX-SAT and TSP.

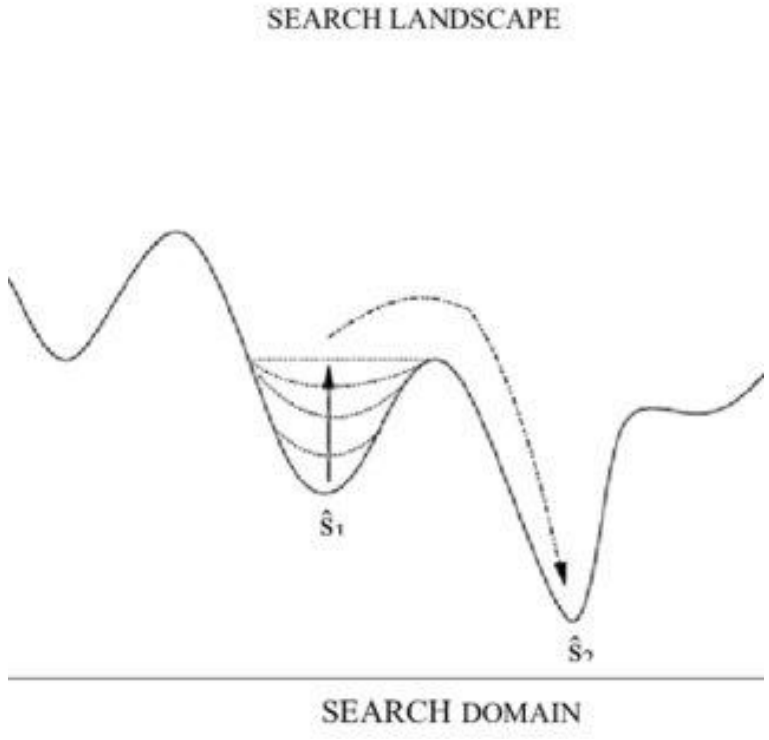
GLS modifies the objective function $f(x)$ by adding a penalty term:

$$f'(x) = f(x) + \lambda \sum_{i \in I} p_i I_i(x)$$

where:

- $f(x)$ is the original objective function.
- λ is a scaling factor.
- p_i is the penalty for feature i.
- $I_i(x)$ is an indicator function that checks if feature i is present in solution x.

Penalties are updated when the search stagnates in a local optimum.

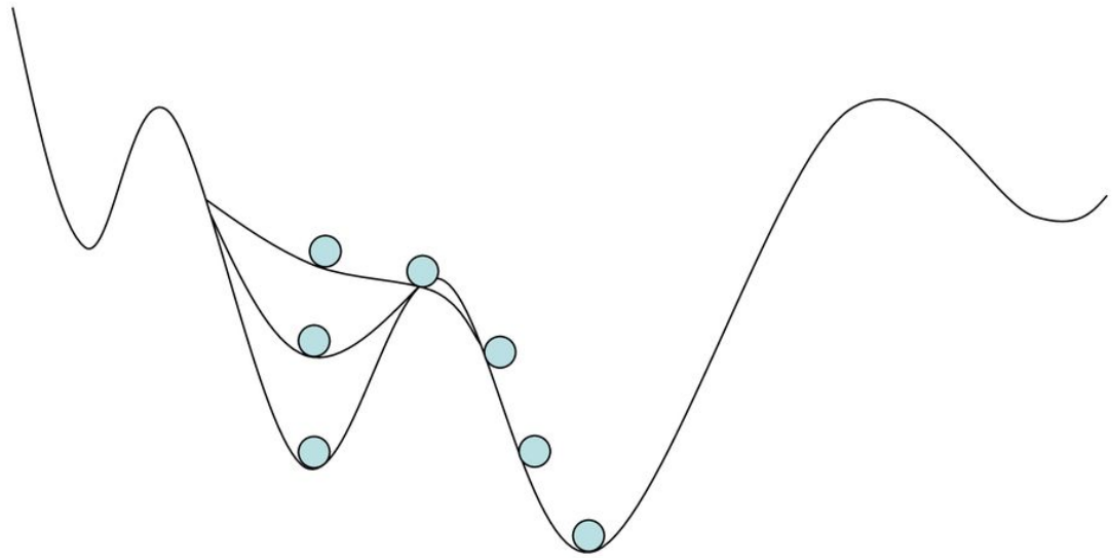


Guided Local Search

- GLS operates by iteratively applying a local search algorithm while penalizing frequently chosen bad solution features, encouraging exploration of new areas in the search space.

1. **Initialize**: Start with an initial solution and compute its objective function value.
2. **Local Search**: Apply a local search heuristic (e.g., Hill Climbing, Simulated Annealing) to find a local optimum.
3. **Penalty Mechanism**: Identify solution features contributing to poor performance and increase their penalties.
4. **Modified Objective Function**: Adjust the objective function by incorporating the penalties.
5. **Repeat**: Continue with local search using the updated objective function until a termination condition is met.

Change the search landscape: Guided Local Search



1.Local Search: Like other local search methods, GLS begins with an initial solution and iteratively explores neighboring solutions to find a local optimum.
2.Penalty Mechanism: GLS introduces a penalty mechanism to discourage revisiting solutions that have already been explored. This helps to escape local optima and promotes diversity in the search process.
3.Guidance: The algorithm uses information about the search history to guide the search toward unexplored areas of the solution space.

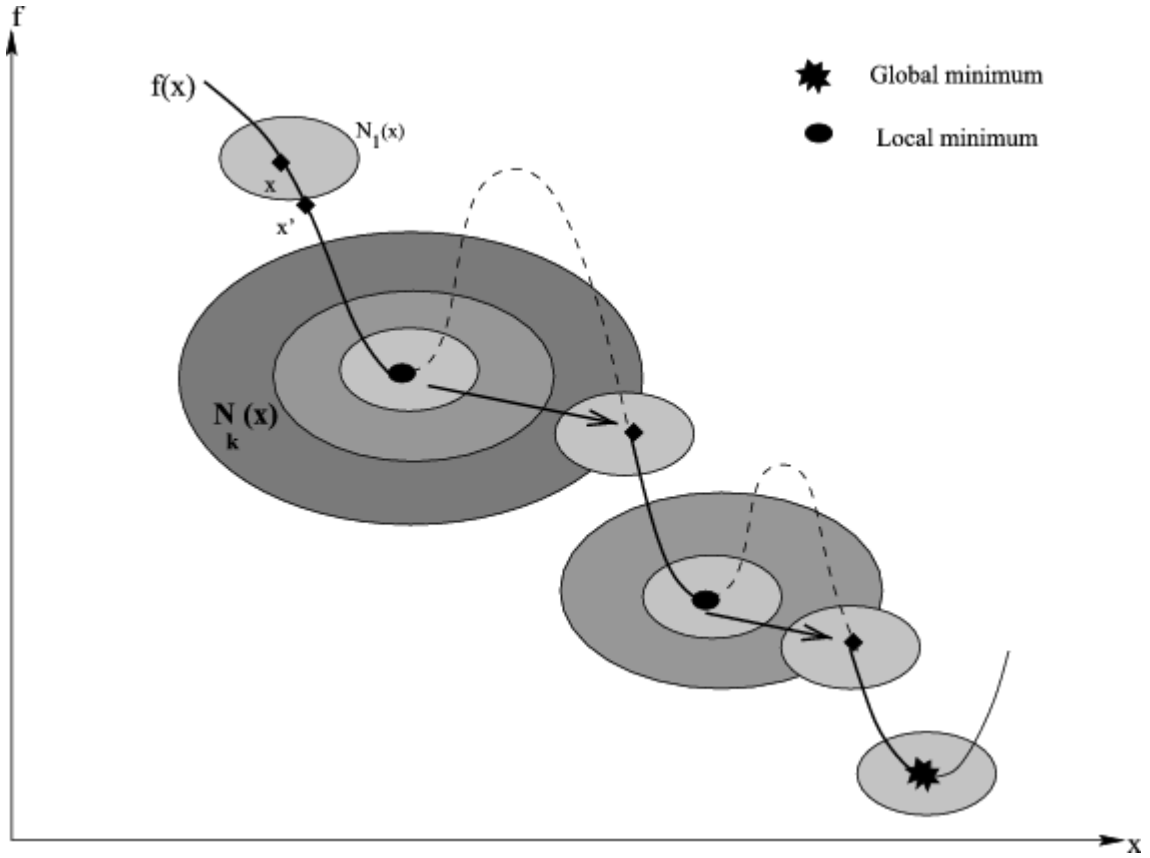
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Variable neighborhood search

- **Variable Neighbourhood Search (VNS)** is a metaheuristic algorithm designed to escape local optima by systematically changing the neighborhood structures during the search process.

1. **Initialize**: Start with an initial solution x and define a set of neighborhood structures $N_1, N_2, ..., N_k$.
2. **Repeat Until Stopping Condition**:
   - Set $k = 1$ (start from the first neighborhood structure).
   - **Shaking**: Randomly perturb x to generate a new solution x' from the k-th neighborhood $N_k(x)$.
   - **Local Search**: Apply a local search heuristic (e.g., Hill Climbing) starting from x', resulting in solution x''.
   - **Move or Not**:
     - If x'' is better than x, update x and reset $k = 1$.
     - Otherwise, increase k to explore the next neighborhood.
   - Repeat until k reaches the maximum allowed neighborhood $k_{\max}$.
3. **Return**: The best solution found.

Change the search landscape: Variable neighborhood search



1. **Greedy Construction**: The algorithm builds a solution incrementally, selecting components based on a greedy criterion, but with a random element that introduces diversity.
2. **Restricted Candidate List (RCL)**: Instead of choosing the best candidate at each step, GRASP maintains a list of the best candidates (RCL). A random choice is made from this list, allowing for exploration of different paths.
3. **Local Search**: After constructing a solution, GRASP applies a local search to refine it. This step aims to improve the solution by exploring its neighborhood.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

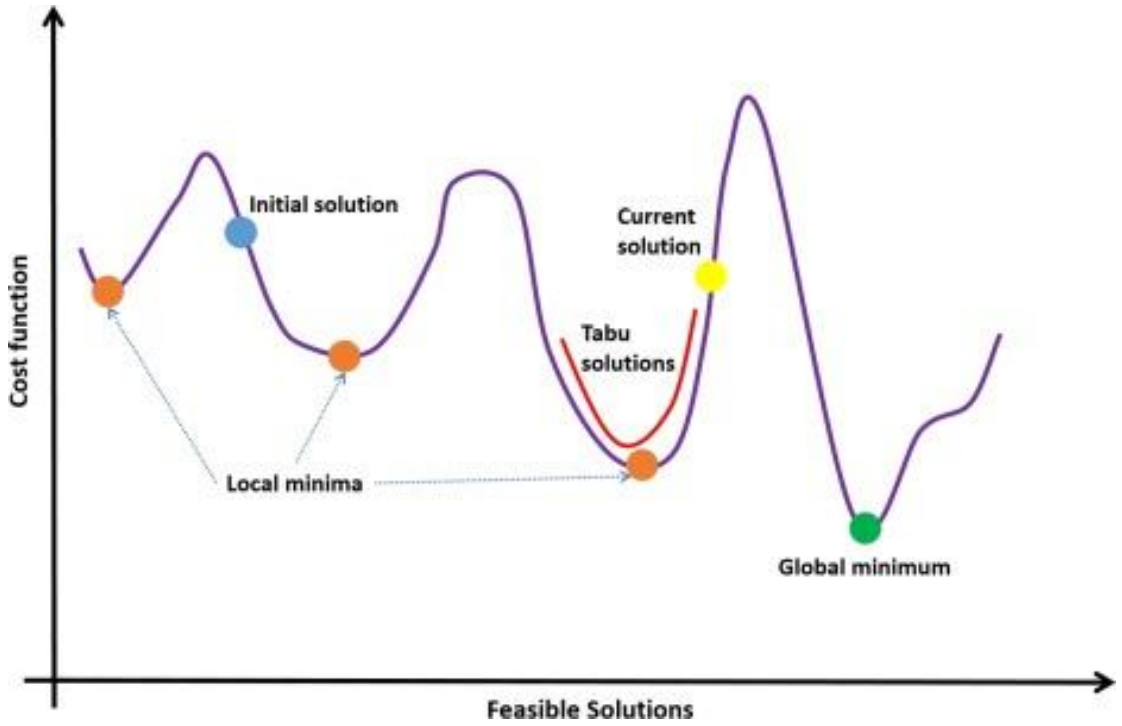
Mechanisms for Escaping from Local Optima II

- Search process (guideline)
- Stopping conditions
- *Mechanism for escaping from local optima*

- Use **Memory** (e.g., **tabu search**)
- **Accept non-improving moves**: allow search using candidate solutions with equal or worse evaluation function value than the one in hand
  - Could lead to long walks on plateaus (neutral regions) during the search process, potentially causing cycles – visiting of the same states
- None of the mechanisms is guaranteed to always escape effectively from local optima

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Use Memory: Tabu Search



Tabu search enhances the performance of local search by relaxing its basic rule. First, at each step worsening moves can be accepted if no improving move is available (like when the search is stuck at a strict local minimum). In addition, prohibitions (hence the term tabu) are introduced to discourage the search from coming back to previously-visited solutions.

1. **Local Search**: Tabu Search starts with an initial solution and explores neighboring solutions to find better candidates.
2. **Tabu List**: A memory structure that keeps track of recently visited solutions or solution components to prevent revisiting them. This list helps avoid local optima.
3. **Aspiration Criteria**: Conditions under which a solution that is normally considered "tabu" (i.e., forbidden) can be accepted if it leads to a sufficiently better solution than the current best.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Termination Criteria (Stopping Conditions) – Examples

- Search process (guideline)
- Stopping conditions
- *Mechanism for escaping from local optima*

- Stop if
  - a *fixed maximum number* of iterations, or moves, objective function evaluations, or a fixed amount of CPU time, is exceeded.
  - the consecutive number of iterations since the last improvement in the best objective function value is *larger than a specified number*.
  - evidence can be given that an *optimum solution has been obtained* – i.e., optimum objective value is known.
  - *no feasible solutions* can be obtained for a fixed number of steps/time – a solution is *feasible* if it satisfies all constraints in an optimisation problem.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Metaheuristics

Local Search

- *[Kirkpatrick, 1983]* Simulated Annealing (SA)
- *[Glover, 1986]* **Tabu Search (TS)**
- *[Voudouris, 1997]* Guided Local Search (GLS)
- *[Stutzle, 1999]* **Iterated Local Search (ILS)**
- *[Mladenovic, 1999]* Variable Neighborhood Search (VNS)

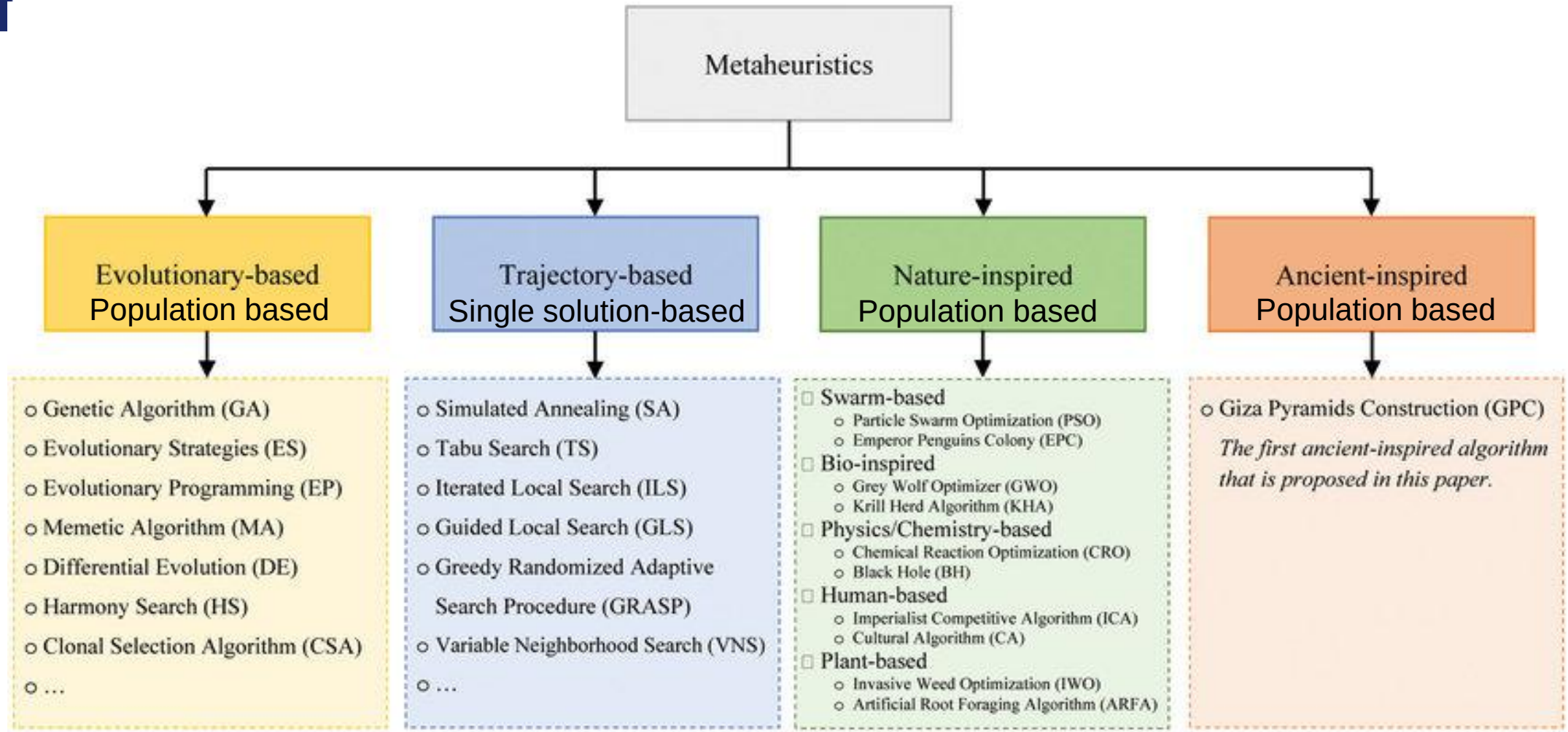
Population-based

- *[Holland, 1975]* Genetic Algorithm (GA)
- *[Smith, 1980]* Genetic Programming (GP)
- *[Goldberg, 1989]* Genetic and Evolutionary Computation (EC)
- *[Moscato, 1989]* Memetic Algorithm (MA)
- *[Kennedy and Eberhart, 1995]* Particle Swarm Optimisation (PSO)

Constructive

- *[[Dorigo, 1992]* Ant Colony Optimisation (ACO)
- *[Resende, 1995]* Greedy Randomized Adaptive Search Procedure (GRASP)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

University of Nottingham
UK | CHINA | MALAYSIA

Local Search Metaheuristics & Iterated Local Search

Stochastic Local Search

- The local search process is started by selecting an initial candidate solution, and then proceeds by iteratively moving from one candidate solution to a neighbouring candidate solution, where the decision on each search step is based on a limited amount of local information only.

```
1: generate initial solution x^c
2: while stopping condition not met do
3:     create new solution x^n = N(x^c)
4:     if f(x^n) ≤ f(x^c) then
5:         x^c = x^n
6:     end if
7: end while
8: return x^c
```

- In **stochastic local search algorithms**, local search algorithms use *randomised* (stochastic) choices in generating and modifying candidate solutions.

Stochastic Local Search

1. **Local Search Component**:
   - SLS algorithms start from an initial solution and explore its neighborhood to find better solutions.
   - Local search is typically deterministic, focusing on refining the current solution.
2. **Stochastic Elements**:
   - Randomization is introduced into the search process to enhance exploration. This can include random moves in the search space or random selection of neighbors.
   - Stochasticity helps to avoid getting stuck in local optima by allowing the algorithm to jump to different areas of the solution space.
3. **Iterative Process**:
   - The algorithm iteratively improves solutions, often using a combination of random restarts and local search refinements.

Stochastic Local Search – Single Point Based Iterative Search (Local Search Metaheuristics)

```
s0 ;  // starting solution
s* = initialise(s0); // e.g., improve s0 or use the same repeat
    // generate a new solution
  s'  = makeMove(s*, memory); // choose a neighbour of s*
  accept = moveAcceptance(s*, s', memory);  // remember s_best
  if  (accept) s* = s'; // else reject new solution s'
until (termination conditions are satisfied);
```

- ***Move Acceptance*** *decides whether to accept or reject the new solution* considering its evaluation/quality, *f*(s'), and memory.
- **Accepting non-improving moves** could be used as a mechanism to escape from local optima

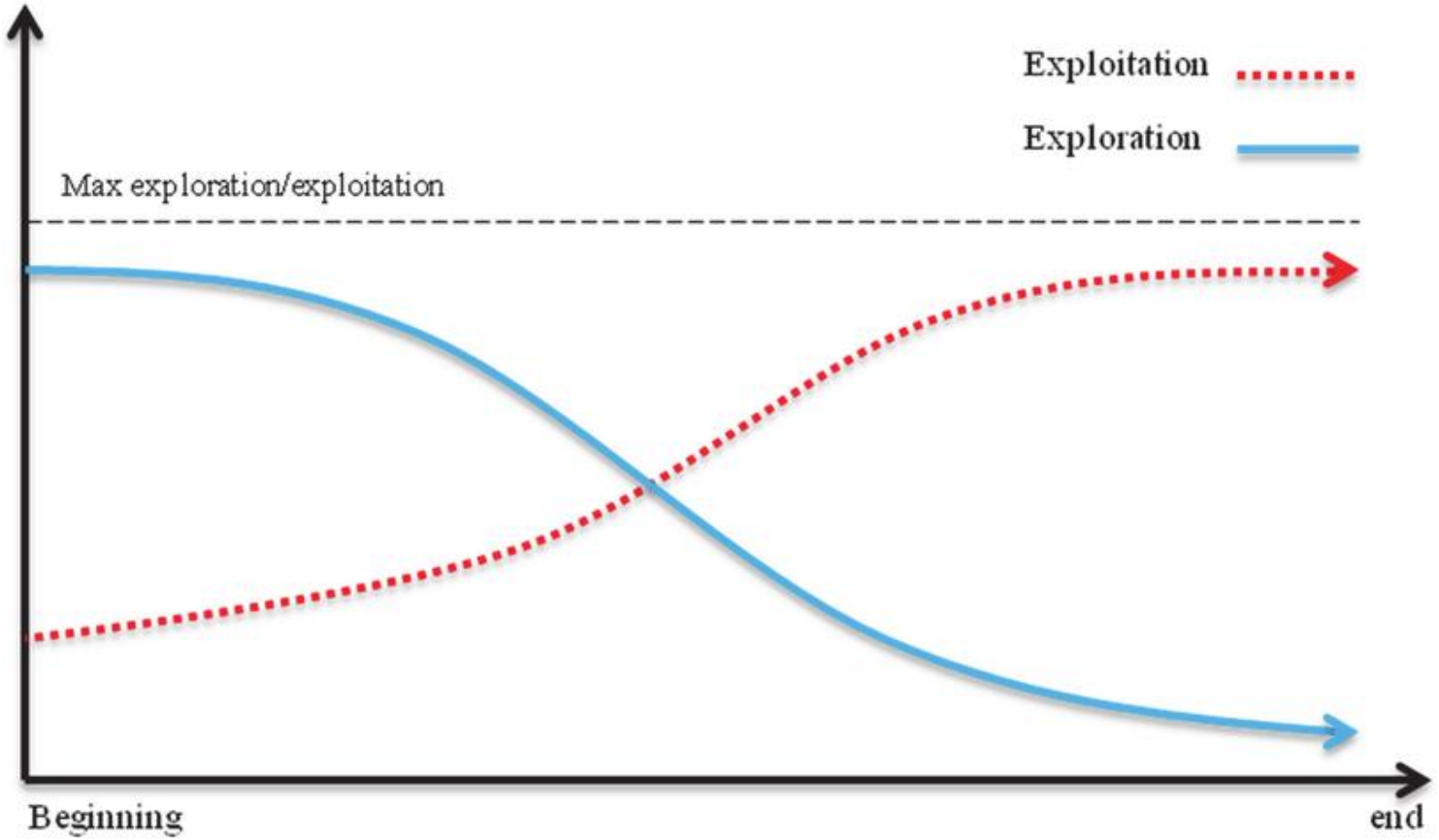
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

The Art of Searching

- Effective search techniques provide a mechanism to balance *exploration* and *exploitation*
  - **Exploitation/intensification** aims to greedily increase solution quality or probability, e.g., by exploiting the evaluation function
  - **Exploration/diversification** aims to prevent stagnation of the search process (getting trapped at a local optimum)
- Aim is to design search algorithms/metaheuristics that can
  - escape local optima
  - balance exploration and exploitation
  - make the search independent from the initial configuration

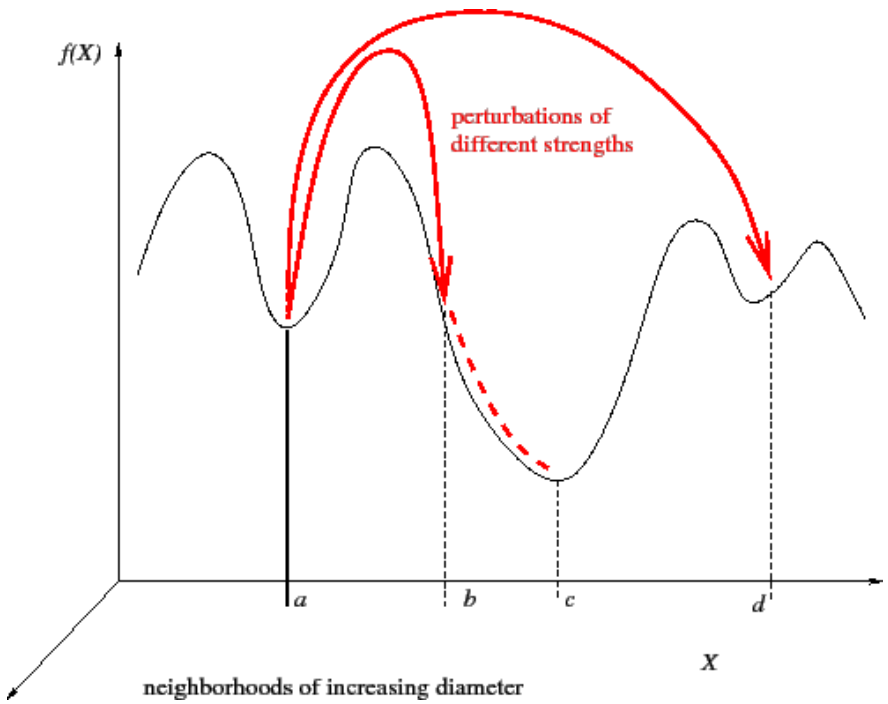
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Exploration vs Exploitation



Iterated Local Search (ILS)

```
s0= GenerateInitialSolution()
//random or construction heuristic
s* = LocalSearch(s0) //not always used
Repeat
   // random move
  s' = Perturbation(s*, memory)
  // hill climbing
  s' = LocalSearch(s' )
  // the conditions that the new local optimum
  // must satisfy to replace the current solution
  s* = AcceptanceCriterion(s*, s', memory) // remember s_best
Until (termination conditions are satisfied)
return s*
```



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Iterated Local Search II

- Based on visiting a sequence of locally optimal solutions by:
  - **perturbing** the current local optimum (exploration/diversification);
  - applying **local search** (exploitation/intensification) after starting from the modified solution.
- A **perturbation** phase might consist of one or more steps
- The perturbation strength is crucial
  - **Too small (weak):** may generate cycles (e.g. imagine a marble that keeps rolling back into the same basin).
  - **Too big (strong):** good properties of the local optima are lost (e.g. imagine a marble that was flung out of a good basin before it could reach the bottom).

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Iterated Local Search III

- Acceptance criteria
  - Extreme in terms of *intensification*: **accept only improving solutions**
  - Extreme in terms of *diversification*: **accept any solution**
  - Other: *deterministic* (like **threshold**), *probabilistic* (like Simulated Annealing)
- Memory
  - restart search if for a number of iterations, no improved solution is found

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Example 1 – ILS for MAX-SAT

- **Generate Initial Solution**: Random
- **Perturbation**: A number of random bit flips (random walk)
- **Local Search**: Use 1-bit-flip neighbourhood, Steepest Descent Hill Climbing
- **Acceptance Criterion**: accept *improving and equal moves* (non-worsening): accept *s' if and only if* $f(s') \leq f(s^*)$

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Iterated Local Search Example (MAX-SAT)

- Problem:
  - $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$
- Representation: Binary string
- Initialisation: random binary string
- Objective function: number of unsatisfied clauses

- **Step 1**: Initialise Solution
  - $s_0 \leftarrow 100100$ (abcdef)
  - $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$
  - $c_0 c_1 c_2 c_3 c_4 c_5$
  - $f(s_0) = 3$
  - (No local search after initialisation)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

$s^* = s_0 = 100100; f(s_0) = 3$

- Problem:
  - $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$
- Representation: Binary string
- Initialisation: random binary string
- Objective function: number of unsatisfied clauses
- **Perturbation Operator: Random bit flip**
- Local Search Operator: Steepest (Gradient) Descent Hill Climbing (SDHC)
- Move Acceptance: Improving or equal ($\leq$)

- **Step 2:** Iterated Local Search (**Loop 1**)
  - Random Bit Flip
    - $s' \leftarrow 110100$
    - Clause SAT: 012345
    - $f(s') = 4$
  - SDHC
    - $f(010100) = 012345 = 3$
    - $f(100100) = 012345 = 3$
    - $\underline{f(111100) = 012345 = 1}$
    - $f(110000) = 012345 = 3$
    - $f(110110) = 012345 = 3$
    - $f(110101) = 012345 = 3$
    - $s' \leftarrow 111100$
  - $1 < 3 \left(f(s') \leq f(s^*)\right)$
    - $\therefore s^* \leftarrow s'$

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

$s^* = s_1 = 111100; f(s_1) = 1$

- Problem:
  - $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$

- **Step 2**: Iterated Local Search (**Loop 2**)
  - Random Bit Flip
    - $s' \leftarrow 101100$
    - Clause SAT: 012345
    - $f(s') = 1$
  - SDHC
    - $f(001100) = 012345 = 2$
    - $f(111100) = 012345 = 1$
    - $f(100100) = 012345 = 3$
    - $\underline{f(101000) = 012345 = 0}$
    - $f(101110) = 012345 = 1$
    - $f(101101) = 012345 = 1$
    - $s' \leftarrow 101000$
  - $0 < 1\ (f(s') \leq f(s^*))$
    - $\therefore s^* \leftarrow s'$
  - return $f(s^*) = 0$;

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Example 2 – Designing Another ILS for MAX-SAT

- **Generate Initial Solution**: Random
- **Perturbation: *intensityOfMutation* times random bit flips**
- **Local Search**: Davis's Bit Hill Climbing for *depthOfSearch* (noOfSteps) times
- **Acceptance Criterion**: accept *improving and equal moves* (non-worsening): accept *s' if and only if f*(*s'*) ≤ *f*(*s**)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Example 2 – Designing Another ILS for MAX-SAT

- **Generate Initial Solution**: Nearest-neighbour
- **Perturbation**: a number of **exchange moves**
  - E.g.,: 1 2 3 4 5 6 → 1 4 3 2 5 6, then 6 4 3 2 5 1
- **Local Search**: first improvement using insertion neighbourhood moves
  - E.g.,: 1 2 3 4 5 6 → 1 3 2 4 5 6, 1 2 3 4 5 6, 1 3 4 2 5 6, 1 3 4 5 2 6, 1 3 4 5 6 2, 2 1 3 4 5 6
- **Acceptance Criterion**: accept *improving and equal moves* (non-worsening): accept *s' if and only if* $f(s') \leq f(s^*)$

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Iterated Local Search (ILS) applied to the Traveling Salesman Problem (TSP)

Problem Setup

- **Cities**: A, B, C, D, E
- **Distance matrix** (symmetric):

City	A	B	C	D	E
A	0	2	9	10	7
B	2	0	6	4	3
C	9	6	0	8	5
D	10	4	8	0	6
E	7	3	5	6	0

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Iterated Local Search (ILS) applied to the Traveling Salesman Problem (TSP)

Step 1: Initial Solution (Greedy Construction)

Start with a greedy method to generate an initial route. Let's choose city **A** as the starting point and visit the nearest unvisited city.

1. **Start at A**.
2. **B** is the nearest city to A (distance = 2).
3. **E** is the nearest city to B (distance = 3).
4. **D** is the nearest city to E (distance = 6).
5. **C** is the remaining city (distance = 8).
6. **Return to A** (distance = 9).

The initial route generated is: **A → B → E → D → C → A**.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Iterated Local Search (ILS) applied to the Traveling Salesman Problem (TSP)

Step 2: Local Search (2-opt)

Now, we perform a local search to improve the current solution by swapping pairs of cities (2-opt). This step tries to reduce the total travel distance.

1. **Initial Route:** A → B → E → D → C → A
2. **Initial Cost:**
   - A → B = 2
   - B → E = 3
   - E → D = 6
   - D → C = 8
   - C → A = 9
   - Total Cost = 2 + 3 + 6 + 8 + 9 = 28

Now, we apply 2-opt swaps to try improving the route.

First 2-opt Swap: Swap E and D (A → B → D → E → C → A)

- **New Route:** A → B → D → E → C → A
- **New Cost:**
  - A → B = 2
  - B → D = 4
  - D → E = 6
  - E → C = 5
  - C → A = 9
  - Total Cost = 2 + 4 + 6 + 5 + 9 = 26

The new route **A → B → D → E → C → A** has a lower cost (26), so we accept this route as the local optimum for this iteration.

Iterated Local Search (ILS) applied to the Traveling Salesman Problem (TSP)

Step 3: Perturbation (Shake)

Now, we apply a perturbation (shake) to the current solution to escape local minima. In this example, we randomly swap two cities in the route.

First Perturbation: Swap B and C (A → C → D → E → B → A)

- **New Route**: A → C → D → E → B → A
- **New Cost**:
  - A → C = 9
  - C → D = 8
  - D → E = 6
  - E → B = 3
  - B → A = 2
  - Total Cost = 9 + 8 + 6 + 3 + 2 = 28

The cost has increased to 28, which is higher than the previous local optimum (26). According to ILS, we may accept this perturbation based on some probability (often depending on temperature or number of iterations).

ACK: Prof. Ender Özcan, UNUK, Dr To

Iterated Local Search (ILS) applied to the Traveling Salesman Problem (TSP)

Step 4: Local Search on Perturbed Solution

Now, we apply local search (2-opt) again on the perturbed solution to refine it.

1. **Current Solution**: A → C → D → E → B → A (Cost = 28)
2. Perform 2-opt swap on this route:
    - Swap C and E: A → E → D → C → B → A
    - **New Cost**:
        - A → E = 7
        - E → D = 6
        - D → C = 8
        - C → B = 6
        - B → A = 2
        - Total Cost = 7 + 6 + 8 + 6 + 2 = 29

This solution has a higher cost, so we discard it and move on.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Ma

Iterated Local Search (ILS) applied to the Traveling Salesman Problem (TSP)

Step 5: Accept or Reject

At this point, we compare the perturbed solution with the best-known solution (local optimum). If the perturbed solution has a better cost or satisfies an aspiration criterion, we accept it; otherwise, we return to the previous best solution.

Since the perturbation led to a cost of 29, which is higher than the current best solution (26), we reject this perturbed solution and keep the previous best route: **A** → **B** → **D** → **E** → **C** → **A** (Cost = 26).

Step 6: Repeat Iterations

We repeat Steps 3–5 for a fixed number of iterations or until no significant improvement occurs. Each time, the local search (2-opt) refines the current solution, and perturbation introduces diversity to explore new regions of the solution space.

ACK: Prof. Ender Özcan, UI

Iterated Local Search (ILS) applied to the Traveling Salesman Problem (TSP)

Step 7: Termination

The algorithm terminates after a predetermined number of iterations (e.g., 100 iterations) or when no further improvement is found. The best solution encountered during the iterations is returned.

Final Solution

After several iterations of perturbation and local search, the final solution might be:

- **Final Route**: A → B → D → E → C → A
- **Final Cost**: 26

This is a local optimum, and if the process is repeated enough times with different initial solutions and perturbations, a near-optimal or global optimum solution can be found.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

ILS – Some Guidelines I

- Initial solution should be to a large extent *irrelevant* for longer runs.
- The interactions among *perturbation strength* and *acceptance criterion* can be particularly important
  - it determines the relative balance of *intensification (exploitation)* and *diversification (exploration)*
  - large perturbations are only useful if they can be accepted.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

ILS – Some Guidelines II

- Advanced **acceptance criteria** take into account search history, e.g., by occasionally reverting to incumbent (past) solution.
- Advanced ILS algorithms may **change nature and/or strength of perturbation adaptively** during search.
- Local search should be as effective and as *fast* as possible. Better local search (speed) generally leads to better ILS performance
- Choose a perturbation operator whose steps cannot be easily undone by the local search

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Iterated Local Search (ILS) vs Random Restart Hill Climbing (RRHC)

- **Multiple Runs of Local Search**: Both involve repeatedly running a local search algorithm (e.g., hill climbing).

- **Escaping Local Optima**: They aim to escape local optima by restarting from new points in the search space.

Iterated Local Search (ILS) vs Random Restart Hill Climbing (RRHC)

Differences:

Feature	Iterated Local Search (ILS)	Random Restart Hill Climbing (RRHC)
How Restarts Are Done	Modifies the previous solution slightly (perturbation) before restarting.	Starts from a completely new random solution.
Memory of Past Solutions	Retains and modifies good solutions.	No memory—each restart is independent.
Efficiency	More efficient because it builds on previous solutions.	Can be less efficient since it doesn't leverage past information.
Diversity of Exploration	Balances exploration and exploitation by perturbing good solutions.	More exploration but lacks a refined exploitation mechanism.

Which One to Use?

- Use **ILS** when s*ma*ll *modifications* of a *good solution* are likely to lead to better results (e.g., combinatorial problems like TSP, SAT, scheduling).

- Use **RRHC** when the landscape has *many local optima* and a fresh random start is more beneficial (e.g., *rugged search spaces* with no clear structure).

Break

University of Nottingham
UK | CHINA | MALAYSIA

Tabu Search

Tabu Search

- Proposed by Fred W. Glover in 1986 and formalised in 1989
- Uses **history (memory structures)** to escape from local minima, inspired by ideas from artificial intelligence in the late 1970s.
- Applies hill climbing/local search
  - Some solution elements/moves are regarded as **tabu (taboo - forbidden)**
  - Proceeds according to the assumption that there is no point in accepting a new (poor) solution unless it is to avoid a path already investigated

Glover F 1986 Future Paths for Integer Programming 2 Glover, F. 1986. Future Paths for Integer Programming and Links to Artificial Intelligence. Computers and Operations Research. Vol. 13, pp. 533-549.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

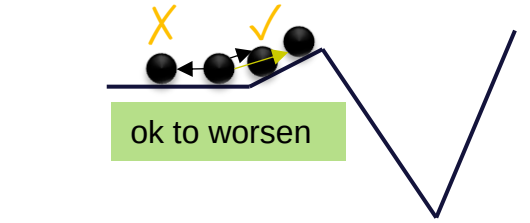
Tabu Search – Fundamentals I

- A stochastic local search algorithm which heavily relies on the use of an **explicit memory** of the search process
  - systematic (and strategic) use of **memory** to guide search process
  - memory typically contains only specific attributes of *previously seen solutions*
  - **simple** tabu search strategies exploit only **short term memory**
  - more **complex** tabu search strategies exploit **long term memory**

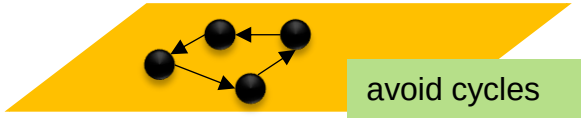
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Tabu Search – Fundamentals II

- In each step, *move to 'non-tabu' best neighbouring solution* (*admissible neighbours*) although it may be *worse than current one*
- To avoid cycles, tabu search tries to *avoid revisiting* previously seen solutions
- Avoid storing complete solutions by basing the memory on *attributes* of recently seen solutions. Example: In a scheduling problem, attributes might include:
  - Pairs of tasks that have been scheduled together.
  - Time slots that have been assigned to specific tasks.



Minimization problem



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Tabu Search – Fundamentals III

- Tabu solution attributes are often defined via local search moves
- **Tabu-list** contains moves which have been made in the recent past
  - What is considered the "past" is determined by the *tabu list length or* **tabu tenure**
- Solutions which contain tabu attributes are *forbidden for a certain number of iterations.*
- Often, an additional *aspiration criterion* (taboo exception) is used:
  - this specifies conditions under which tabu status may be overridden (e.g., if the considered step leads to improvement in incumbent solution).

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Short-term vs Long-term Memories

- The two memory structures work together to balance **local intensification** and **global diversification**:
- **Short-Term Memory:**
  - Prevents immediate reversals and cycles.
  - Focuses on improving the current solution by avoiding recently made moves.
- **Long-Term Memory:**
  - Guides the search away from over-explored regions by penalizing frequently visited moves or encouraging rarely explored ones.
  - Can intensify search efforts in promising areas by revisiting elite solutions.
- These memories can either be implemented **physically (separate data structures)** or **logically (a single structure with different roles and time scales)**, depending on the problem and implementation.

Example: Tabu Search for MAX-SAT I

- **Initialisation**: random – select an assignment randomly from set of all truth assignments
- **Neighbourhood**: 1-bit flip neighbourhood
- **Memory**: Associate tabu status (Boolean value) with each variable in given conjunctive normal form (CNF).
  - variables are tabu IFF they have been changed in the last T *steps;* (T is the *tabu tenure* or *tabu list size*)
  - for each variable x_i store the number of the search step when its value was last changed t_i; x_i is tabu IFF $c - t_i < T$, where c is the current search step number.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Example: CNF & CNF File Format

CNF Representation

In this case, the CNF formula consists of three clauses:

- C_1: $x_1 \vee x_2$
- C_2: $\neg x_1 \vee x_3$
- C_3: $x_2 \vee \neg x_3$

Corresponding CNF File Format

Now, let's convert this CNF formula into the CNF file format:

basic Copy

```
p cnf 3 3
1 2 0
-1 3 0
2 -3 0
```

Example: CNF & CNF File Format

Breakdown of the CNF File Format

1. **Header**:
    - `p cnf 3 3`:
        - `p` indicates that this is a problem line.
        - `cnf` indicates that the formula is in conjunctive normal form.
        - The first `3` is the number of variables (here, x_1, x_2, x_3).
        - The second `3` is the number of clauses.

2. **Clauses**:
    - Each subsequent line represents a clause, with literals followed by `0` to signify the end of the clause:
        - `1 2 0`: Represents clause $C_1 = (x_1 \vee x_2)$.
        - `-1 3 0`: Represents clause $C_2 = (\neg x_1 \vee x_3)$.
        - `2 -3 0`: Represents clause $C_3 = (x_2 \vee \neg x_3)$.

Tabu Search for MAX-SAT II

- **Termination (Stopping Criteria)**: upon finding a truth assignment which satisfies all clauses or exceeding a maximum number of search steps.
- Tabu used to be state of the art for SAT solving.
- Crucial for efficient implementation (read/write tabu list):
  - keep time complexity of search steps minimal by using special data structures, incremental updating and caching mechanism for evaluation function values;
  - efficient determination of tabu status

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

An iteration of Tabu Search for MAX-SAT

- Problem (minimisation):
  - $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$
- Neighbourhood: 1-bit flip
- Assume T = 2
- Current iteration (c_i) is 1207
- Current list content: [1205, 59, 1206, 11, 0, 928]
  - This means that 1st bit (a) was flipped in the 1205th step, second bit (b) was flipped in the 59th step, and so on…

$$s^* = 101100; f(s^*) = 1$$

- Consider all 1-bit flip neighbours which are not in the tabu list
  - $(\mathbf{001100})$ **X** $\boldsymbol{tabu}$ $(\mathbf{1207 - 1205 \leq 2}$
  - $f(111100) = 012345 = 1$ (1 clause unsatisfied)
  - $(\mathbf{100100})$ **X** $\boldsymbol{tabu}$ $(\mathbf{1207 - 1206 \leq 2}$
  - $f(101000) = 012345 = 0$ (0 clause unsatisfied)
  - $f(101110) = 012345 = 1$ (1 clause unsatisfied)
  - $f(101101) = 012345 = 1$ (1 clause unsatisfied)
  - $s' \leftarrow 101000$
- update tabu list: [1205, 59, 1206, 1207, 0, 928]

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Explanation

1. **Neighborhood**: The neighbors are all solutions obtained by flipping a single bit in the current solution $s^* = 101100$.
2. **Tabu List**: Solutions corresponding to recent flips are considered tabu. The tabu list is $[1205, 59, 1206, 1207, 0, 928]$, where the index represents the iteration when the bit flip occurred. A flip is tabu if the difference between the current iteration ($ci = 1207$) and the tabu iteration is $\leq T = 2$.
3. **Objective Function $f(s)$**: The value of $f(s)$ for each candidate solution is calculated, representing how well the solution satisfies the MAX-SAT clauses.
4. **Tabu Tenure**: A candidate solution is skipped if it is tabu, unless it satisfies an aspiration criterion.

An iteration of Tabu Search for MAX-SAT

- Problem (minimisation):
  - $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$
- Neighbourhood: 1-bit flip
- Tabu tenure = 2
- Current iteration (*ci*) is 1207
- Current tabu list content: <1, 3> (tail)
  - This means that 1st bit (a) was flipped two steps ago, while 3rd bit (c) was flipped in the immediate previous step.

$$s^* = 101100; f(s^*) = 1$$

- Consider all 1-bit flip neighbours which are not in the tabu list
  - $(\mathbf{0}01100)$ **X** $tabu$
  - $f(1\mathbf{1}1100) = 012345 = 1$
  - $(10\mathbf{0}100)$ **X** $tabu$
  - $\underline{f(101\mathbf{0}00) = 012345 = 0}$
  - $f(1011\mathbf{1}0) = 012345 = 1$
  - $f(10110\mathbf{1}) = 012345 = 1$
  - $s' \leftarrow 101000$
- update list: <3, 4> (tail)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Non-tabu Candidate Clause Evaluation

- The truth assignment s=101000 satisfies the most clauses (f(101000)=6) and is therefore the best candidate among the given solutions

1. $s = 111100$ ($a = 1, b = 1, c = 1, d = 1, e = 0, f = 0$):
   - Clause 1: $a \vee b = 1 \vee 1 = 1$ **(satisfied)**
   - Clause 2: $\neg d \vee f = 0 \vee 0 = 0$ **(not satisfied)**
   - Clause 3: $\neg a \vee c = 0 \vee 1 = 1$ **(satisfied)**
   - Clause 4: $b \vee \neg f = 1 \vee 1 = 1$ **(satisfied)**
   - Clause 5: $\neg b \vee c = 0 \vee 1 = 1$ **(satisfied)**
   - Clause 6: $c \vee e = 1 \vee 0 = 1$ **(satisfied)**

   $f(111100) = 5$ **(5 clauses satisfied)**

2. $s = 101000$ ($a = 1, b = 0, c = 1, d = 0, e = 0, f = 0$):
   - Clause 1: $a \vee b = 1 \vee 0 = 1$ **(satisfied)**
   - Clause 2: $\neg d \vee f = 1 \vee 0 = 1$ **(satisfied)**
   - Clause 3: $\neg a \vee c = 0 \vee 1 = 1$ **(satisfied)**
   - Clause 4: $b \vee \neg f = 0 \vee 1 = 1$ **(satisfied)**
   - Clause 5: $\neg b \vee c = 1 \vee 1 = 1$ **(satisfied)**
   - Clause 6: $c \vee e = 1 \vee 0 = 1$ **(satisfied)**

   $f(101000) = 6$ **(all 6 clauses satisfied)**

Non-tabu Candidate Clause Evaluation

- The truth assignment s=101000 satisfies the most clauses (f(101000)=6) and is therefore the best candidate among the given solutions

3. $s = 101110$ ($a = 1, b = 0, c = 1, d = 1, e = 1, f = 0$):

- Clause 1: $a \vee b = 1 \vee 0 = 1$ **(satisfied)**
- Clause 2: $\neg d \vee f = 0 \vee 0 = 0$ **(not satisfied)**
- Clause 3: $\neg a \vee c = 0 \vee 1 = 1$ **(satisfied)**
- Clause 4: $b \vee \neg f = 0 \vee 1 = 1$ **(satisfied)**
- Clause 5: $\neg b \vee c = 1 \vee 1 = 1$ **(satisfied)**
- Clause 6: $c \vee e = 1 \vee 1 = 1$ **(satisfied)**

$f(101110) = 5$ **(5 clauses satisfied)**

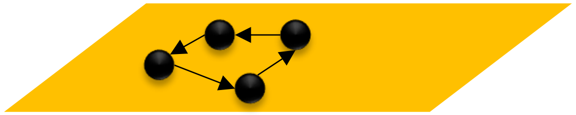
4. $s = 101101$ ($a = 1, b = 0, c = 1, d = 1, e = 0, f = 1$):

- Clause 1: $a \vee b = 1 \vee 0 = 1$ **(satisfied)**
- Clause 2: $\neg d \vee f = 0 \vee 1 = 1$ **(satisfied)**
- Clause 3: $\neg a \vee c = 0 \vee 1 = 1$ **(satisfied)**
- Clause 4: $b \vee \neg f = 0 \vee 0 = 0$ **(not satisfied)**
- Clause 5: $\neg b \vee c = 1 \vee 1 = 1$ **(satisfied)**
- Clause 6: $c \vee e = 1 \vee 0 = 1$ **(satisfied)**

$f(101101) = 5$ **(5 clauses satisfied)**

Practical Considerations

- Appropriate choice of tabu tenure critical for performance
- Tabu tenure: the length of time/number of steps t for which a move is forbidden
  - t too low - risk of cycling



  - t too high - may restrict the search too much
  - t =7 has often been found sufficient to prevent cycling
  - $t = \sqrt{n}$
  - number of tabu moves: 5 – 9
- If a tabu move is smaller (assuming minimization problem) than the *aspiration level* then we accept the move (use of aspiration criteria to override tabu status)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

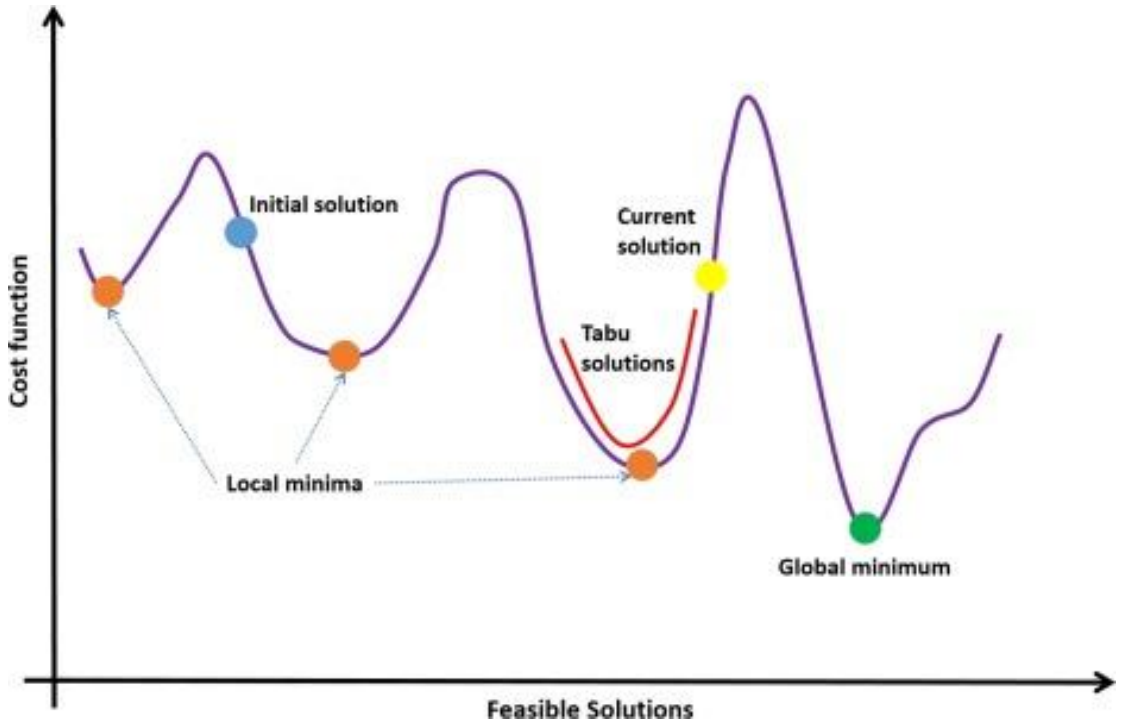
Examples of aspiration criteria

- **Aspiration criteria** in Tabu Search are rules that override the tabu status of a move if the move is considered "exceptionally good" or beneficial to the search process. These criteria are used to ensure that the algorithm does not exclude potentially promising solutions simply because they are tabu.
  - Improving the Current Solution
  - Improving the Best Known Solution
  - Diversification Goals
  - Satisfying Hard Constraints
  - Long-Term Gains
  - Breaking Cycles
  - Achieving a Target Threshold / Metric
  - User-Defined Rules

Example

1. **Current Best Solution**: Let's say the current best solution has a value of 10 (aspiration level).
2. **Tabu Move**: A move results in a new solution with a value of 8, but this move is currently tabu.
3. **Applying Aspiration Criteria**: Since 8 is less than the aspiration level of 10, the algorithm overrides the tabu status and accepts the move.

In Summary: Tabu Search



Tabu search enhances the performance of local search by relaxing its basic rule. First, at each step worsening moves can be accepted if no improving move is available (like when the search is stuck at a strict local minimum). In addition, prohibitions (hence the term tabu) are introduced to discourage the search from coming back to previously-visited solutions.

1. **Local Search**: Tabu Search starts with an initial solution and explores neighboring solutions to find better candidates.
2. **Tabu List**: A memory structure that keeps track of recently visited solutions or solution components to prevent revisiting them. This list helps avoid local optima.
3. **Aspiration Criteria**: Conditions under which a solution that is normally considered "tabu" (i.e., forbidden) can be accepted if it leads to a sufficiently better solution than the current best.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

University of Nottingham
UK | CHINA | MALAYSIA

Introduction to Scheduling

Scheduling

- **Scheduling** deals with the *allocation of resources to tasks over given time periods* and its goal is to optimise one or more objectives. The resources and tasks in an organisation can take many different forms.
- It is a decision-making process that is used on a regular basis in many manufacturing and services industries.
- Many real-world applications including
  - Project planning
  - Semiconductor manufacturing
  - Gate assignment at airports
  - Scheduling tasks in CPUs/parallel computers, ...

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Scheduling – Framework and Notation

Scheduling is the process of planning, controlling and optimising work and workloads in a production/manufacturing process.

machines $i = 1,\ldots,m$ **jobs** $j = 1,\ldots,n$

(i, j) processing step, or operation, of job j on machine i

Job characteristics/data

Processing time p_{ij} - processing time of job j on machine i

Release date r_j - earliest time at which job j can start its processing (start)

Due date d_j - committed shipping or completion (due) date of job j (end)

Weight w_j - importance of job j relative to the other jobs in the system

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Types of Job Scheduling

- **Single Machine Scheduling**: Scheduling jobs on a *single machine* to minimize total completion time, tardiness, or other criteria.
- **Parallel Machine Scheduling**: Scheduling jobs on *multiple machines*, where jobs can be processed on any available machine.
- **Open Shop Scheduling (O)**: Jobs consist of multiple tasks, but the *sequence* in which the tasks are processed on machines is *not fixed*.
- **Flow Shop Scheduling (F)**: Jobs pass through *a series of machines in a specific order* (e.g., job A goes to machine 1, then machine 2).
- **Job Shop Scheduling (J)**: Jobs have specific routing requirements through machines, making it more complex than flow shop scheduling.

Multi-stage problem: O, F, J

```
Jobs:
  Job 1: M1, M2, M3
  Job 2: M2, M1, M3
  Job 3: M3, M1, M2

Machines: M1, M2, M3
```

Open Shop

```
Jobs:
  Job 1: M1 → M2 → M3
  Job 2: M1 → M2 → M3
  Job 3: M1 → M2 → M3

Machines: M1, M2, M3
```

Flow Shop

```
Jobs:
  Job 1: M1 → M3 → M2
  Job 2: M2 → M1 → M3
  Job 3: M3 → M2 → M1

Machines: M1, M2, M3
```

Job Shop

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Classification of Scheduling Problems – Notation

- **Scheduling problem**: $\alpha \mid \beta \mid \gamma$
  - α machine characteristics
  - β job characteristics
  - γ optimality criteria

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Sample Machine Characteristics (α)

This refers to the capabilities and limitations of the machines or resources involved in the scheduling process.

Single-stage problem (vs. Multi-stage problem: O, F, J)

1 **Single machine**

P_m **Identical machines in parallel**

- *m* machines in parallel
- Job *j* requires a single operation and may be processed on any of the *m* machines

Q_m **Machines in parallel with different speeds**

R_m **Unrelated machines in parallel**

- machines have different speeds for different jobs

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Sample Machine Characteristics (α)

- This refers to the capabilities and limitations of the machines or resources involved in the scheduling process.
- **Examples**:
  - **Number of Machines**: How many machines are available for processing jobs (single vs. parallel machines).
  - **Machine Speed**: The processing speed of each machine, which can affect how quickly jobs are completed.
  - **Setup Times**: The time required to prepare a machine for a new job, which may impact scheduling decisions.
  - **Machine Types**: Different types of machines may have specific functions or constraints (e.g., different technologies or capacities).

Single-Stage vs Multi-Stage

Feature	Single-Stage	Multi-Stage (O, F, J)
Stages	Single processing stage	Multiple processing stages
Complexity	Simple, low complexity	Higher complexity, depends on type
Job Flow	No routing; all jobs processed on one machine	Jobs may follow predefined or unique sequences
Machines Involved	Single machine or workstation	Multiple machines or workstations
Scheduling Goal	Optimize metrics like makespan or tardiness	Optimize across all stages (e.g., makespan)
Examples	Printing jobs on one printer	Assembly lines (F), custom jobs (J)
Solution Approach	Simple algorithms (SJN, FCFS, EDD)	Advanced heuristics, metaheuristics

ACK: Prof. Ender

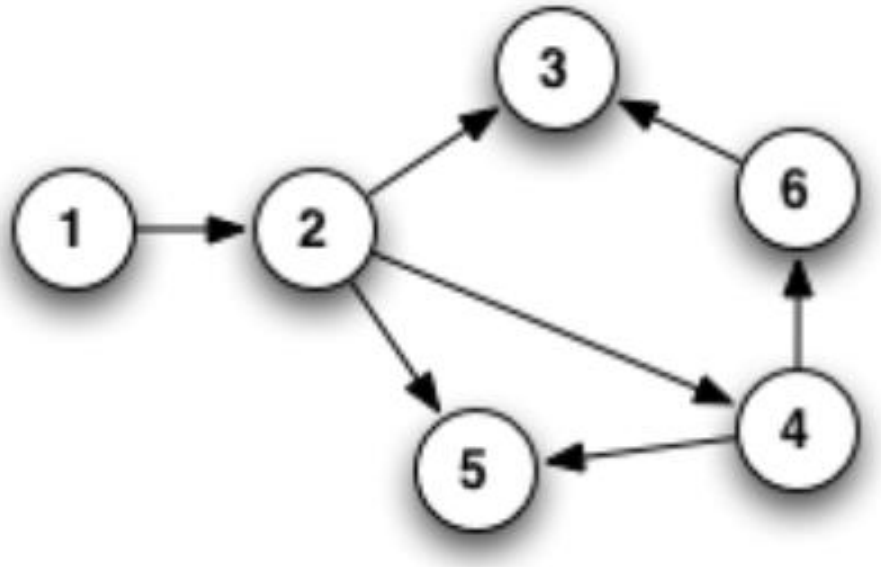
Sample Job Characteristics (β)

- This encompasses the properties and requirements of the jobs to be scheduled.
- The processing time may be different, or equal for all jobs, or even of unit length. All processing times are assumed to be integers.
- **Processing time** p_{ij} - processing time of job *j* on machine *i* (if a single machine then p_j)
- **Due date** d_j - committed shipping or completion (due) date of job *j*
- **Weight** w_j - importance of job *j* relative to the other jobs in the system

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Sample Job Characteristics (β) II

- **Release date** r_j - earliest time at which job *j* can start its processing
- **Precedence** *prec* – Precedence relations might be given for the jobs. If *k* precedes *l*, then starting time of *l* should be not earlier than completion time of *k*
- **Sequence dependent setup times** s_{jk} - setup time between jobs *j* and *k*
- **Breakdowns** *brkdwn* - machines are not continuously available



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Sample Job Characteristics (β)

- This encompasses the properties and requirements of the jobs to be scheduled.
- **Examples**:
  - **Job Duration**: The time required to complete each job, which affects overall scheduling.
  - **Job Due Dates**: Deadlines by which jobs need to be completed, which can influence priority.
  - **Job Precedence**: Some jobs may need to be completed before others (dependencies).
  - **Job Priority**: Certain jobs may be deemed more critical and require higher priority in scheduling.

Sample Optimality Criteria (γ)

We define for each job *j*:

- C_{ij} **completion time** of the operation of job *j* on machine *i*
- C_j time when job *j* exits the system
- $L_j = C_j - d_j$ **lateness** of job *j*
- $T_j = \max(C_j - d_j, 0)$ **tardiness** of job *j*
- $U_j = \begin{cases} 1 & \text{if } C_j > d_j \\ 0 & \text{otherwise} \end{cases}$ unit penalty of job *j*

This refers to the goals or objectives that the scheduling process aims to achieve.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Lateness (L_j) vs Tardiness (T_j)

Lateness (L_j)

- **Formula**:

$$L_j = C_j - d_j$$

where:

- C_j = Completion time of job j
- d_j = Due date of job j

- **Definition**:
  - Lateness measures the difference between the completion time of a job and its due date. It can be positive, negative, or zero:
    - **Positive Lateness**: The job is completed after its due date (late).
    - **Negative Lateness**: The job is completed before its due date (early).
    - **Zero Lateness**: The job is completed exactly on its due date.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Lateness (L_j) vs Tardiness (T_j)

Tardiness (T_j)

- **Formula**:

$$T_j = \max(C_j - d_j, 0)$$

- **Definition**:
  - Tardiness measures only the amount of time a job is late. It is always zero or positive:
    - **Positive Tardiness**: Indicates how late the job is (if $C_j > d_j$).
    - **Zero Tardiness**: Indicates that the job is completed on or before its due date (if $C_j \leq d_j$).

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Lateness (L_j) vs Tardiness (T_j)

Feature	Lateness (L_j)	Tardiness (T_j)
Value Range	Can be positive, negative, or zero	Always zero or positive
Interpretation	Measures how early or late a job is	Measures only how late a job is
Use Case	Provides a broader view of job performance	Focuses specifically on late jobs

Sample Optimality Criteria (γ)

- This refers to the goals or objectives that the scheduling process aims to achieve.
- **Examples**:
  - **Minimizing Makespan**: The total time required to complete all jobs from start to finish.
  - **Minimizing Tardiness**: Reducing the delays of jobs beyond their due dates.
  - **Maximizing Resource Utilization**: Ensuring that machines and resources are used efficiently.
  - **Balancing Workloads**: Distributing jobs evenly across available machines to avoid bottlenecks.

Recap: Notation

- **<u>Scheduling problem</u>**: $\alpha \mid \beta \mid \gamma$
  - α machine characteristics
  - β job characteristics
  - γ optimality criteria

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Scheduling Notation - Examples

- $1 \mid prec \mid C_{max}$
  A single machine, general precedence constraints, minimising makespan (maximum completion time).
- $P3 \mid d_j, s_{jk} \mid \sum L_j$

  3 identical machines, each job has a due date and sequence dependent setup times between jobs, minimising total lateness of jobs.
- $R \mid\mid \sum C_j$

  variable number of unrelated parallel machines, no constraints, minimising total completion time.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

A Single Machine Scheduling Problem

$$1|\, d_j|\, \sum w_j T_j$$

- Given *n* jobs to be processed by a single machine, each job (*j*) with a *due date* (d_j) (i.e., hard deadline), processing time (p_j), and a weight (w_j) (i.e., the job with the highest weight is more important and so needs to finish on time),
- **find the optimal sequencing of jobs** producing the minimal weighted *tardiness* (T_j).
- Tardiness is 0 if a job completes on time ($C_j \leq d_j$), otherwise it is the time spent after the due date to completion ($C_j - d_j$)

$T_j = \max(C_j - d_j, 0)$
tardiness of job *j*

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Example: Computing Weighted Tardiness

Given:

- 3 jobs with the following parameters:

Job i	Processing Time (p_i)	Due Date (d_i)	Weight (w_i)
1	4	6	3
2	2	5	2
3	3	7	5

Example: Computing Weighted Tardiness

Step 1: Calculate Completion Times (C_i)

We need to determine the order of job processing. For simplicity, we assume the order: **1 → 2 → 3**.

Job i	Start Time	Completion Time (C_i)
1	0	4
2	4	6
3	6	9

Step 2: Compute Tardiness (T_i)

Using the formula $T_i = \max(0, C_i - d_i)$:

Job i	Completion Time (C_i)	Due Date (d_i)	Tardiness (T_i)
1	4	6	0
2	6	5	1
3	9	7	2

Example: Computing Weighted Tardiness

Step 3: Compute Weighted Tardiness ($w_i \cdot T_i$)

Job i	Weight (w_i)	Tardiness (T_i)	Weighted Tardiness ($w_i \cdot T_i$)
1	3	0	0
2	2	1	2
3	5	2	10

Step 4: Total Weighted Tardiness

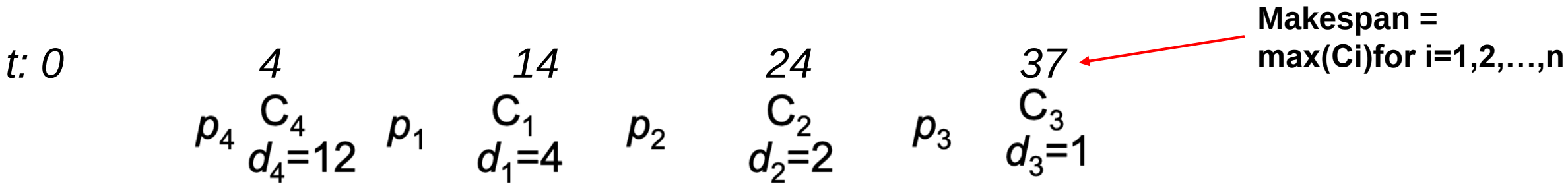
$$\text{Total Weighted Tardiness} = \sum_{i=1}^{n} w_i \cdot T_i = 0 + 2 + 10 = 12$$

Example 1: Computing Weighted Tardiness

$1|\ dj\ |\ \sum w_j T_j$

jobs	1	2	3	4
p_i	10	10	13	4
d_i	4	2	1	12
w_i	14	12	1	12

What would be weighted tardiness of the solution which uses the shortest processing time for sequencing the jobs? (E.g., <4, 1, 2, 3>)



$\sum w_j T_j = w_4 \max(C_4 - d_4, 0) + w_1 \max(C_1 - d_1, 0) + w_2 \max(C_2 - d_2, 0) + w_3 \max(C_3 - d_3, 0)$

$= 12\max(-8,0) + 14\max(10,0) + 12\max(22,0) + 1\max(36,0)$

$= 0 + 140 + 264 + 36 = 440$

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Illustration of a Run on a Scheduling

Job characteristics/data

Processing time p_{ij} - processing time of job *j* on machine *i*

Release date r_j - earliest time at which job *j* can start its processing

Due date d_j - committed shipping or completion (due) date of job *j*

Weight w_j - importance of job *j* relative to the other jobs in the system

Example:

jobs	1	2	3	4
p_j	10	10	13	4
d_j	4	2	1	12
w_j	14	12	1	12

$\sum w_j T_j$

Schedule four jobs on a machine

$T_j = \max(C_j - d_j, 0)$
tardiness of job *j*

Neighbourhood operator: go through all schedules that can be obtained through adjacent pairwise interchanges, choose the best.

Tabu-list: pairs of jobs (*j*, *k*) that were swapped within the last two moves

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Tabu Search Algorithm for Scheduling Example

```
1  l=0; s_l ← initialize, maxTabuSize = 2;
2  S_best ← s0
3  tabuList ← []
4  while (not stoppingCondition())
5      candidateList ← []
6      bestCandidate ← null
7      for (sCandidate in sNeighborhood) // any configuration in the neighbourhood of s: sCandidate ∈ N(s_l)
8          if ( (not tabuList.contains(sCandidate)) and  ( F(sCandidate) < F(bestCandidate) ) )
9              bestCandidate ← sCandidate; bestMove ← (i, j) // swapped adjacent jobs
10         end
11     end
12     l++; s_l ← bestCandidate
13     if  ( F(bestCandidate) < F(S_best) )
14          S_best ← bestCandidate
15     end
16     tabuList.push( reverse(bestMove) ); // save (j, i) into tabu list
17     if (tabuList.size > maxTabuSize)
18         tabuList.removeFirst()
19     end
20 end
21 return S_best
```

Absolute best

Best in each iteration

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Illustration of a run

jobs	1	2	3	4
p_j	10	10	13	4
d_j	4	2	1	12
w_j	14	12	1	12

max(10-2,0)

max((10+10)-4,0)

S_0 = <2, 1, 4, 3>

$F(S_0) = \sum w_j T_j = 12 \cdot 8 + 14 \cdot 16 + 12 \cdot 12 + 1 \cdot 36 = 500 = F(S_{best})$

max((10+10+4)-12,0)

max((10+10+4+13)-1,0)

F(<1, 2, 4, 3>) = 480

F(<2, 4, 1, 3>) = 436 = $F(S_{best})$

F(<2, 1, 3, 4>) = 652

Tabu-list: { (1, 4) }

$T_j = \max(C_j - d_j, 0)$
tardiness of job j

S_1 = <2, 4, 1, 3>, $F(S_1)$ = 436

F(<4, 2, 1, 3>) = 460

F(<2, **1, 4**, 3>) (= 500) tabu!

F(<2, 4, 3, 1>) = 608

Tabu-list: { (2, 4), (1, 4) }

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Example 1 Runs

Step 1: Initialize

- Let the initial sequence of jobs be $s_0 = [1, 2, 3, 4]$.
- Compute the objective function $F(s_0)$: $\sum w_i T_i$.
- Initialize an empty Tabu list.

Step 2: Calculate $F(s_0)$

Using the initial sequence:

- Completion times (C_i):
  $C_1 = 10, C_2 = 20, C_3 = 33, C_4 = 47.$
- Tardiness (T_i):
  $T_1 = \max(10 - 4, 0) = 6,$
  $T_2 = \max(20 - 2, 0) = 18,$
  $T_3 = \max(33 - 1, 0) = 32,$
  $T_4 = \max(47 - 12, 0) = 35.$
- Weighted tardiness:
  $F(s_0) = 14(6) + 12(18) + 1(32) + 12(35) = 84 + 216 + 32 + 420 = 752.$

Example 1 Runs

Step 2: Calculate $F(s_0)$

Using the initial sequence:

- Completion times (C_i):
  $C_1 = 10, C_2 = 20, C_3 = 33, C_4 = 47.$
- Tardiness (T_i):
  $T_1 = \max(10 - 4, 0) = 6,$
  $T_2 = \max(20 - 2, 0) = 18,$
  $T_3 = \max(33 - 1, 0) = 32,$
  $T_4 = \max(47 - 12, 0) = 35.$
- Weighted tardiness:
  $F(s_0) = 14(6) + 12(18) + 1(32) + 12(35) = 84 + 216 + 32 + 420 = 752.$

Step 3: Generate Neighborhood and Select Best Candidate

Perform pairwise adjacent swaps to generate new sequences:

1. Swap $1 \leftrightarrow 2$: Sequence $[2, 1, 3, 4]$.
2. Swap $2 \leftrightarrow 3$: Sequence $[1, 3, 2, 4]$.
3. Swap $3 \leftrightarrow 4$: Sequence $[1, 2, 4, 3]$.

Calculate F for each sequence and choose the best non-Tabu sequence.

Example 1 Runs

Step 4: Update Tabu List

Add the selected move (swap) to the Tabu list and update the best solution.

Let me compute these values.

Results:

- **Initial Sequence:** $[1, 2, 3, 4]$
  Weighted tardiness: $F(s_0) = 752$.
- **Neighbors (generated by swapping adjacent jobs):**
  1. Swap $3 \leftrightarrow 4$ → Sequence: $[1, 2, 4, 3]$, Weighted tardiness: 610.
  2. Swap $1 \leftrightarrow 2$ → Sequence: $[2, 1, 3, 4]$, Weighted tardiness: 772.
  3. Swap $2 \leftrightarrow 3$ → Sequence: $[1, 3, 2, 4]$, Weighted tardiness: 898.

Best Neighbor:

The best neighbor is $[1, 2, 4, 3]$ with a weighted tardiness of 610.

Example 1 Runs

Next Steps:

1. Update the current solution to $[1, 2, 4, 3]$.
2. Add the move $(3, 4)$ to the Tabu list.
3. Repeat the process until a stopping condition is met (e.g., no further improvement or maximum iterations).

Improving Tabu Search

- Further improvements can be achieved by using *intermediate-term* or *long-term memory* to achieve additional *intensification* or *diversification*.
  - backtrack to elite candidate solutions, reset tabu list
  - freeze certain solution components for a number of steps
  - occasionally force rarely used solution components
  - extend evaluation function to capture frequency of use of candidate solutions (or components)
  - or other heuristics…

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

University of Nottingham
UK | CHINA | MALAYSIA

Tabu Search example for the Traveling Salesman Problem (TSP)

Tabu Search example for the Traveling Salesman Problem (TSP)

Step 1: Problem Setup

Objective:

Find the shortest route to visit all cities exactly once and return to the starting city.

Example Problem:

We have 5 cities with the following distances (in a symmetric matrix):

City	A	B	C	D	E
A	0	2	9	10	7
B	2	0	6	4	3
C	9	6	0	8	5
D	10	4	8	0	6
E	7	3	5	6	0

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Tabu Search example for the Traveling Salesman Problem (TSP)

Step 2: Initial Solution

Generate Initial Route:

Randomly generate an initial route:

$$\text{Route: A} \rightarrow \text{B} \rightarrow \text{D} \rightarrow \text{E} \rightarrow \text{C} \rightarrow \text{A}$$

Calculate the Cost:

Sum the distances for the route:

$$\text{Cost} = A \rightarrow B(2) + B \rightarrow D(4) + D \rightarrow E(6) + E \rightarrow C(5) + C \rightarrow A(9) = 26$$

Initial Route: $\text{A} \rightarrow \text{B} \rightarrow \text{D} \rightarrow \text{E} \rightarrow \text{C} \rightarrow \text{A}, \text{Cost} = 26$

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Tabu Search example for the Traveling Salesman Problem (TSP)

Step 3: Generate Neighbors

Define Neighbors:

Neighbors are generated by swapping two cities in the route (excluding the start/end city A).

Example: Swap B and E in the current route:

$$\text{New Route: A} \rightarrow \text{E} \rightarrow \text{D} \rightarrow \text{B} \rightarrow \text{C} \rightarrow \text{A}$$

Calculate Cost for New Route:

$$\text{Cost} = A \rightarrow E(7) + E \rightarrow D(6) + D \rightarrow B(4) + B \rightarrow C(6) + C \rightarrow A(9) = 32$$

Other Neighbors:

- Swap D and C: $\text{A} \rightarrow \text{B} \rightarrow \text{C} \rightarrow \text{E} \rightarrow \text{D} \rightarrow \text{A}$, Cost = 28
- Swap B and C: $\text{A} \rightarrow \text{C} \rightarrow \text{D} \rightarrow \text{E} \rightarrow \text{B} \rightarrow \text{A}$, Cost = 27

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Tabu Search example for the Traveling Salesman Problem (TSP)

Step 4: Select Best Neighbor

Choose the Best Neighbor:

From the neighbors, pick the one with the smallest cost that is not in the tabu list:

- $\mathrm{A} \rightarrow \mathrm{C} \rightarrow \mathrm{D} \rightarrow \mathrm{E} \rightarrow \mathrm{B} \rightarrow \mathrm{A}, \mathrm{Cost} = 27$

Step 5: Update Tabu List

Add Current Solution to Tabu List:

- Add the current route ($A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A$) to the tabu list.

Tabu List:

Maintain a fixed size for the tabu list (e.g., 3 solutions). If it exceeds the size, remove the oldest entry.

Tabu List after Iteration 1:

$$\text{Tabu List: } [\mathrm{A} \rightarrow \mathrm{B} \rightarrow \mathrm{D} \rightarrow \mathrm{E} \rightarrow \mathrm{C} \rightarrow \mathrm{A}]$$

ACK: Prof. Ender Özcan, UNUK, E

Tabu Search example for the Traveling Salesman Problem (TSP)

Step 6: Repeat Iterations

Repeat Steps 3–5 for a fixed number of iterations or until convergence.

Iteration 2:

1. **Current Solution**: $\mathrm{A} \to \mathrm{C} \to \mathrm{D} \to \mathrm{E} \to \mathrm{B} \to \mathrm{A}, \mathrm{Cost} = 27$
2. **Generate Neighbors**:
   - Swap D and E: $\mathrm{A} \to \mathrm{C} \to \mathrm{E} \to \mathrm{D} \to \mathrm{B} \to \mathrm{A}, \mathrm{Cost} = 25$
   - Swap C and B: $\mathrm{A} \to \mathrm{B} \to \mathrm{D} \to \mathrm{E} \to \mathrm{C} \to \mathrm{A}, \mathrm{Cost} = 26$ (in tabu list, skip)
3. **Select Best Neighbor**: $\mathrm{A} \to \mathrm{C} \to \mathrm{E} \to \mathrm{D} \to \mathrm{B} \to \mathrm{A}, \mathrm{Cost} = 25$
4. **Update Tabu List**: Add $\mathrm{A} \to \mathrm{C} \to \mathrm{D} \to \mathrm{E} \to \mathrm{B} \to \mathrm{A}$.

Tabu List after Iteration 2:

$$\text{Tabu List: } [\mathrm{A} \to \mathrm{B} \to \mathrm{D} \to \mathrm{E} \to \mathrm{C} \to \mathrm{A}, \mathrm{A} \to \mathrm{C} \to \mathrm{D} \to \mathrm{E} \to \mathrm{B} \to \mathrm{A}]$$

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Tabu Search example for the Traveling Salesman Problem (TSP)

Step 7: Termination

After a fixed number of iterations (e.g., 100) or when no significant improvement occurs:

Final Solution:

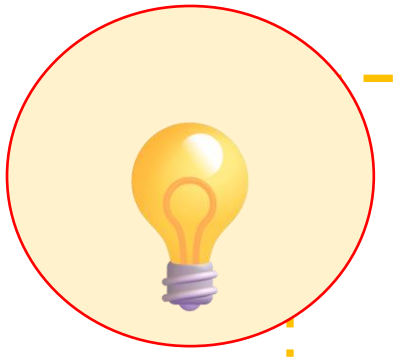
- **Optimal Route**: $\mathrm{A} \rightarrow \mathrm{C} \rightarrow \mathrm{E} \rightarrow \mathrm{D} \rightarrow \mathrm{B} \rightarrow \mathrm{A}$
- **Cost**: 25

Summary of Iterations

Iteration	Current Route	Best Neighbor	Cost	Tabu List
1	$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A$	$A \rightarrow C \rightarrow D \rightarrow E \rightarrow B \rightarrow A$	27	$[A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A]$
2	$A \rightarrow C \rightarrow D \rightarrow E \rightarrow B \rightarrow A$	$A \rightarrow C \rightarrow E \rightarrow D \rightarrow B \rightarrow A$	25	$[A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A, A \rightarrow C \rightarrow D \rightarrow E \rightarrow B \rightarrow A]$

ACK: Prof. Ender Özcan, UNUK,

Summary



Each metaheuristic has a mechanism for **escaping from local optima**

- Balance between **exploration** and **exploitation** is important
- **Iterated Local Search** enforces exploration and exploitation explicitly
- **Tabu Search** uses flexible memory structures in conjunction with strategic restrictions and aspiration levels

There are different types of metaheuristics embedding different components

- The performance of a metaheuristic often varies **based on the chosen design components** – hence the need for empirical studies (experiments)

Scheduling contains many crucial NP hard real-world optimisation problems often dealt with in manufacturing/ production using heuristics/hyper/metaheuristics.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Learning Outcomes

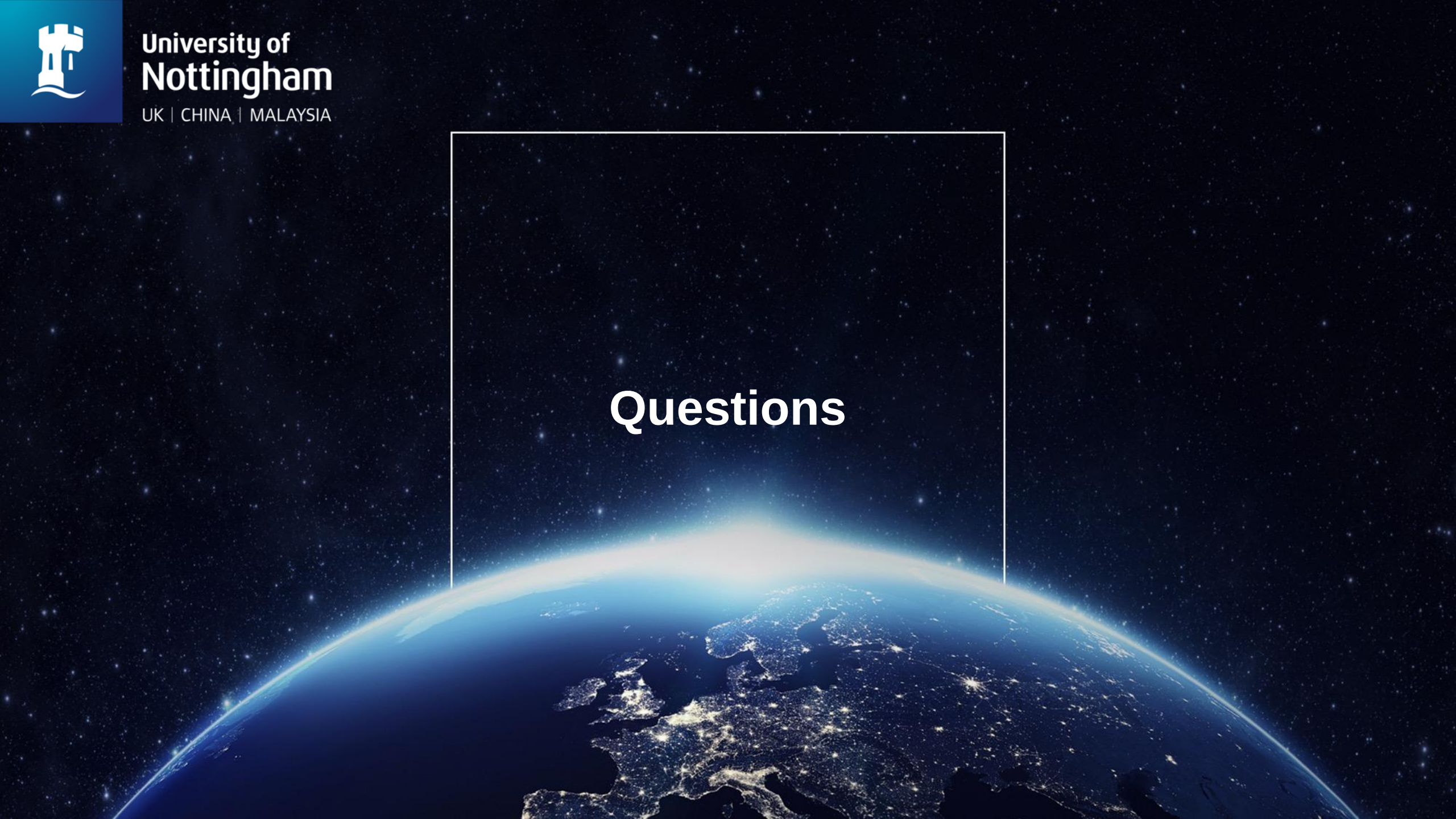
IDENTIFY

1. Metaheuristics
2. Local Search Metaheuristics and Iterated Local Search
3. Tabu Search
4. Scheduling

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

University of Nottingham
UK | CHINA | MALAYSIA

Questions



References

- Metaheuristic Algorithms for Optimization: A Brief Review https://www.mdpi.com/2673-4591/59/1/238

- An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges https://link.springer.com/article/10.1007/s10462-023-10470-y

- https://en.wikipedia.org/wiki/List_of_metaphor-based_metaheuristics


University of
Nottingham
UK | CHINA | MALAYSIA
NEXT:
Move Acceptance